Requests of a Feather Must Flock Together

Anonymous Author(s)

Abstract

Current organization of the paper:

- **Title** - the following things should come up in the title in some form - homogenity, LLM serving, shared prefix workloads, chunked hash tree, RL, feather.
- **Abstract** - highlight about LLM inference and shared prefix workloads, key challenges, ideas, along with numbers of throughput improvements
- **Introduction**
- **Background**
- **Motivation** - if possible, add actual dram read throughput numbers
- **Design and Implementation**
- **Evaluation** - final results on all the metrics - throughput and different types of latencies. after this, observations about the results (why and in what all sweet zones we are much much better). Important - present DRAM related numbers for naive vllm and after we did our changes. Present average batch size numbers too. Also highlight stuff related to kv cache eviction, which is already done in batchllm, for example. run on other gpus and add results in appendix if lack of space. compare with PAT on a cloud GPU. also for different models to compare stuff. comparison when we remove some kind of component from our approach? for example, what if we use token radix tree instead of chunked hash tree, can present cycles taken by individual functions, along with throughput improvements. What if we don't stop the batch at the right time - low workload, we will have hetero batches, which will perform worse. What if we don't have an RL model - we are not falling back to fcfs, in low request rate settings, our batch sizes would be much much smaller, and hence throughput might be slightly lower (basically both micro-benchmarks and ablation studies).
- **Conclusions** - brief summary of the insights, design and results achieved. what are the future steps in these directions?
- **Appendix** - can move unncecessary algorithms, family load request queue as the theoretical baseline which helped us improve? Any kind of not-so-important results. f1-f2 heatmap plots.

Rough proposed page limits after shrinking for the final draft:

- Abstract and Intro: 1.5 pages
- Background: 1 page
- Motivation: 2.5 - 3 pages
- Design and Implementation: 2.5 - 3 pages
- Eval: 3 pages (including micro-benchmarks and ablation studies)
- Conclusion: 0.5 pages

1 Introduction

need to add 1-2 images?

Large language models (LLMs) [13, 27, 36, 37] have been increasingly deployed across various domains, achieving remarkable performance in applications including conversational assistants [11, 14], code generation [16], and retrieval-augmented generation (RAG) [20, 40]. They have started to profoundly impact our work and daily routines. Many cloud companies [5, 15, 26] are racing to provide these applications as hosted services. However, running these applications is very expensive, requiring a large number of hardware accelerators such as GPUs. As deployment grows, the latency and resource efficiency of both online and offline inference have become increasingly critical. In response, a plethora of systems have been deployed to optimize different aspects of LLM serving, such as scheduling strategies [2, 29, 46, 48], parallelization techniques [33, 38], memory management [18, 32, 43], quantization [10, 23], and kernel-level performance [28, 42].

To understand the landscape of optimizations covered in this paper, we briefly review the key technical foundations. LLM inference operates in two distinct phases: a compute-bound **prefill** phase that processes all prompt tokens in parallel to produce the first output token, and a memory-bound **decode** phase that generates one token per iteration. The decode phase relies on a **KV cache** — stored keys and values from prior tokens — which grows with context length and increases the memory footprint.

At the hardware level, GPUs execute thousands of parallel threads across Streaming Multiprocessors (SMs), each with local shared memory and tensor cores, all backed by a shared L2 cache and global memory; the interplay between compute throughput and memory bandwidth at this level shapes the design of nearly every optimization discussed in this paper. this paragraph is not really needed in intro, can be easily skipped.

Mismatched compute profiles of prefill and decode motivate a large body of work on **batching and scheduling**. Systems have explored strategies that either interleave [2], disaggregate [48], or spatially co-schedule [17, 24] the two phases.

Many practical workloads, such as few shot prompting [9, 31], system prompts [1], and RAG [21], also exhibit shared prompt prefixes across requests. Reusing the KV cache of

these shared prefixes avoids redundant computation and reduces memory traffic. Systems such as SGLang [45] and vLLM [19] manage this through radix trees and Merkle-tree-inspired hashing schemes, respectively.

A complementary line of kernel-level work [28, 39, 41, 42] pushes this idea further by restructuring how queries are grouped and KV data is loaded within the attention kernel itself, enabling the reuse of shared KV data and reducing redundant global memory traffic at the GPU execution level.

Despite these advances, a precise empirical understanding of *why* and *how* shared prefix structure affects decode throughput has been lacking. In this paper, we present a series of controlled experiments that systematically quantify the relationship between prefix structure and inference performance.

Our central finding is that decode-phase throughput is primarily governed by KV cache memory access patterns rather than raw arithmetic compute. (should we reword the previous statement?) Intuitively, during the decode phase, every new token generated by a request requires attending to *all* previously seen tokens, which means that the GPU must repeatedly read through the entire KV cache at every generation step. When multiple requests in a batch share the same prompt prefix, they traverse the same regions of GPU memory at nearly the same time. This synchronized access improves both *spatial locality* (the hardware prefetcher can reliably anticipate and load upcoming memory regions) and *temporal locality* (a memory block fetched for one request is likely to be reused by a neighbouring request before being evicted). Together, these effects increase L2 cache hit rates and sustain high memory bandwidth utilization.

We validate this hypothesis through several controlled experiments. First, we show that even a single request, out of a large batch, with a different prefix, can reduce decode throughput by nearly 2×. This stark sensitivity arises because the GPU memory access stream becomes interleaved across two disjoint KV cache regions, disrupting prefetcher confidence and reducing temporal cache reuse. Second, we demonstrate that throughput improves smoothly and monotonically as the fraction of shared prefix tokens increases: moving from no sharing to full sharing yields over 60% higher throughput. Third, we find that the position of sharing within a sequence also matters; since all requests traverse the KV cache in lockstep during decode, sharing at the beginning of a sequence provides the greatest locality benefit. Once requests diverge, synchronized memory access is disrupted, and subsequent sharing in later parts of the sequence provides insignificant returns. Fourth, beyond locality benefits, we show that when the number of simultaneously active prefixes becomes large enough to exhaust GPU memory capacity, the system is forced to evict and recompute KV cache blocks, causing an additional sharp decline in throughput and effective batch size. This highlights that prefix-aware scheduling must account for both memory access locality and GPU memory capacity constraints.

We further characterize the interplay between batch size and prefix structure. Increasing batch size initially improves throughput by better utilizing GPU tensor cores; however, once compute resources are saturated, adding more requests only increases KV cache traffic without proportional throughput gains, making memory bandwidth the bottleneck. Under end-to-end execution, including both prefill and decode phases, we show that prefix-homogeneous batches can outperform larger heterogeneous batches once decode lengths are sufficiently long. This is because each decode step requires another full sweep of the KV cache, allowing the locality benefits of aligned memory access to compound over time. Interestingly, even a homogeneous batch of 50 requests can surpass a heterogeneous batch of 800 requests when decode lengths are long enough, demonstrating that memory locality can outweigh the loss in tensor core utilization.

Finally, we examine the cost of detecting prefix sharing at runtime. Existing systems such as SGLang [45] use radix-tree-based schedulers that perform token-by-token comparisons, dynamic node splitting, and repeated tree traversals to identify shared prefixes. We show that for a workload of 2000 requests with 10k-token prefixes, these scheduling policies incur CPU overhead of 78-89 seconds, which is on the same order as the GPU execution time itself. This reveals a fundamental inefficiency: the very mechanism used to exploit prefix sharing can itself become a bottleneck, motivating a shift towards a lightweight prefix identification scheme.

should the metric come out here?

Motivated by these insights, we propose a scheduling system built around two complementary components. The first is the **Chunked Hash Tree (CHT)**, a lightweight data structure for fast prefix detection. Rather than token-by-token radix tree traversals, CHT represents each request as a compact vector of cumulative prefix hashes computed at fixed token intervals. A min-heap ordered by per-request missing counts - the number of prefix chunks not yet covered by the active batch - enables $O(\log W)$ identification of the best prefix-sharing candidate among W waiting requests. On the top of this, lazy heap updates keep the total CPU overhead minimal, even for thousands of long-prompt requests.

The second component addresses the question of when to stop adding requests to a batch. Growing a batch improves GPU utilization, but each new request with a diverging prefix shrinks the overall shared prefix length and reduces locality. We frame this stop-or-continue decision as a reinforcement learning problem. At each step of batch construction, the scheduler observes the current batch size, the shared prefix length before and after admitting the next candidate, and the number of waiting requests that share the same prefix level (info about all the state variables not needed probably). Tabular policies, trained online from observed decode throughput, learn to balance these competing objectives adaptively

across different GPU configurations and workload conditions, which a fixed heuristic policy cannot do reliably. Together, the CHT and the learned batching policy form an end-to-end scheduling system that simultaneously reduces CPU scheduling overhead and improves decode-phase throughput by constructing large, prefix-homogeneous batches.

add eval results once ready!

1.1 Contributions of this paper

In summary, the paper makes the following contributions:

- We conduct an in-depth analysis of the bottlenecks that lead to reduced DRAM bandwidth utilization for prefix-shared workloads.
- We also identify several CPU-level overheads while detecting prefix sharing in a batch of requests.
- We design and implement an efficient data structure that effectively organizes requests and reduces the scheduler's CPU overhead for such workloads.
- We come up with a metric that, in turn, drives our batching decisions. We also employ reinforcement learning policies to guide batch formation.
- Experimental evaluations on real-world workloads show that we outperform state-of-the-art inference engines in both latency and throughput while achieving significantly better DRAM bandwidth utilization.

1.2 Overall paper organization

We present related work in this field in §2. We provide an extensive set of experiments that demonstrate our insights and key ideas in §3. We next present the design and implementation of our ideas in §4. We evaluate our implementation with real workloads against state-of-the-art baselines in §5, along with micro-benchmarks and an ablation study, followed by the conclusion in §6.

2 Background

add delta of each section from our work?

2.1 LLM Inference and Attention

LLM inference operates auto-regressively: each output token is generated from the hidden state of the preceding token. Every request passes through two phases. In the **prefill** phase, all prompt tokens are processed in parallel to produce the first output token; its latency is called *time-to-first-token* (TTFT). In the **decode** phase, one token is generated per iteration, with per-token latency called *time-between-tokens* (TBT). Prefill is compute-bound due to high token parallelism, while decode is memory-bound due to sequential generation against a growing context.

Each decode step involves three major computations: Self-Attention, Query-Key-Value-Output (QKVO) projection, and a Multi-Layer Perceptron (MLP). Self-attention models global token dependencies as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V} \tag{1}$$

where d is the head dimension. During the decode phase, the query $\mathbf{Q}$ for the current token attends to all prior keys $\mathbf{K}$ and values $\mathbf{V}$. Since $\mathbf{K}$ and $\mathbf{V}$ are reused across steps, they are cached in GPU global memory as the **KV cache**, eliminating redundant recomputation at the cost of an increasing memory footprint. These two characteristics of LLM inference—the asymmetry between the prefill and decode phases and the memory pressure of the KV cache—motivate much of the systems work discussed below. To delve deeper into this, we first explain how GPUs operate and then present a case study of existing work.

need to add time spent in attention and so on?

2.2 GPU Execution Model

what all things to mention? The GPU's hardware is arranged in a hierarchy that supports execution at a scale of hundreds of thousands of parallel threads. The main processing unit of a GPU is a Streaming Multiprocessor (SM), with modern GPUs containing around a hundred SMs. Each SM has an L1 cache and shared memory, along with tensor cores for accelerated general matrix multiplication (GEMM) and execution units for integer and floating point operations. The shared memory is a user-addressable partition of the L1 cache. The GPU memory is accessed by SMs through the shared L2 cache. mention about prefetcher here, and cite something? Is there some abrupt break in the flow just after this?

2.3 Batching and Scheduling for LLM Inference

A central challenge in LLM serving is that prefill and decode iterations have different compute profiles, making it difficult to mix them in a batch without interference. vLLM [19] laid the groundwork for flexible batching by introducing *PagedAttention*, which stores the KV cache in non-contiguous (reviewers might target us on this, basically if they are physically non-contiguous, then how are the benefits coming in? we need to clarify that initial allocations all contiguous.) fixed-size memory blocks analogous to virtual memory paging, eliminating fragmentation and enabling KV blocks to be shared across requests. Building on this, Sarathi Serve [2] addresses the throughput-latency tradeoff more directly by splitting large prompts into fixed-size chunks via *chunked prefills* and using *stall-free batching* to coalesce these chunks with ongoing decode operations, eliminating the generation stalls caused by long prefill iterations blocking active decodes. Where Sarathi-Serve mitigates interference by interleaving, DistServe [48] eliminates it entirely by disaggregating prefill and decode into separate GPUs, independently optimizing resource allocation and parallelism for each phase. POD-Attention [17] takes a different approach, keeping both phases on the same GPU while introducing a

fused attention kernel with SM-aware CTA scheduling to overlap their compute and memory-bound operations concurrently on the same multiprocessor. Similarly, Bullet [24] exploits spatial-temporal GPU resource sharing between prefill and decode, coupling an SLO-aware scheduler with a dynamic SM resource manager to balance throughput and latency objectives without physical separation. bullet is one of the latest papers, so nice to describe, but not too much relevant contribution?

Beyond phase co-scheduling, several systems focus on request-level batching policies to improve efficiency. BucketServe [46] reduces padding waste by grouping requests with similar sequence lengths into buckets, improving GPU memory efficiency. Pang et al. [30] takes a runtime-adaptive approach, dynamically adjusting batch size based on live GPU memory usage and latency SLA constraints. For offline settings, BlendServe [44] introduces a resource-aware prefix tree and a dual-scanner scheduling algorithm that reorders and co-schedules requests with complementary compute and memory demands to maximize GPU utilization. Collectively, these works highlight that batching and scheduling decisions, whether at the phase, request, or resource level, have a decent impact on overall inference efficiency.

2.4 Prefix Sharing in LLM Serving

Many real-world workloads, such as few-shot prompting, system prompts, and retrieval-augmented generation (cite for each category, or if done in the intro and not needed here) involve requests that share a common prompt prefix. Reusing the KV cache of shared prefixes avoids redundant computation and reduces memory pressure, but doing so efficiently requires careful coordination between the serving system and memory management. SGLang [45] was among the first to formalize this, introducing RadixAttention, which uses a radix tree to identify and reuse KV caches across requests with shared prefixes in a multi-call serving setting. Radix trees explicitly store token sequences in a hierarchical structure where common prefixes share the same path. We show a simple example of a radix tree in Figure 1.

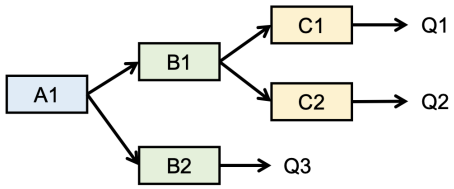


Figure 1. An example of a radix tree from [28], where queries Q1, Q2 share A1, B1 sub-sequences with each other, and only A1 with query Q3.

vLLM [19] also implements automatic prefix caching [34] using a hashing scheme inspired by Merkle trees, as shown in Figure 2. Each KV-cache block is assigned a hash that depends on both the tokens inside the block and the hash of the preceding prefix block. This effectively forms a chain of hashes representing the entire prefix. If two requests produce the same sequence of hashes, the system can freely reuse the corresponding KV-cache blocks.

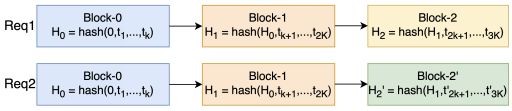


Figure 2. An example of vllm Merkle Tree.

BatchLLM [47] extends the idea of prefix sharing to large offline batches, explicitly grouping requests by shared prefixes for global KV cache reuse, then reordering them and applying memory-aware token batching to improve the mix of prefill and decode tokens. BlendServe [44] similarly leverages a resource-aware prefix tree, but pairs it with a scheduling algorithm that jointly optimizes for both prefix reuse and complementary compute-memory demand balancing. Together, these systems establish that effective prefix sharing requires not just identifying common prefixes but also carefully scheduling the order in which requests are processed to maximize KV cache hit rates and memory efficiency.

2.5 Prefix-Aware Attention Kernels

While the systems above handle prefix sharing at the serving and scheduling level, a complementary line of work focuses on exploiting shared prefix structure directly inside the attention kernel, reducing redundant global memory traffic at the GPU execution level. PAT [42] takes a direct approach by packing queries that share the same prefix into a single CTA, so the shared KV cache is loaded from global memory once and broadcast across all grouped queries. It introduces a pack-forward-merge pipeline with a prefix-aware pack scheduler, multi-tile attention kernels, and multi-stream execution to improve hardware utilization. DEFT [39] addresses the same problem in tree-structured decoding by grouping queries according to their shared KV prefixes (KV-guided grouping), ensuring each prefix segment is fetched exactly once; it further introduces Flattened Tree KV Splitting to balance KV partitions across thread blocks for uniform GPU utilization. ChunkAttention [41] approaches the problem from a memory layout perspective, storing KV caches in a prefix tree of chunked KV tensors and introducing a two-phase partition kernel, chunk-first and sequence-first, to improve memory locality and parallelism when operating over shared prefix chunks. (improve chunk attention description) FastTree [28] targets radix-tree-structured workloads specifically, grouping queries that share the same tree context to reuse KV

cache directly from shared memory, reducing global memory pressure and better utilizing tensor cores. Across all these works, the common insight is that the key to efficient prefix-aware attention lies in restructuring how queries are grouped and how KV data is loaded.

3 Motivation

3.1 Experiment Setup and Overview

Our central hypothesis is that **decode-phase** throughput is largely determined by the memory access pattern of the KV cache rather than by compute. In particular, when all the requests in a batch share a common prefix, their attention operations traverse the same KV cache regions at roughly the same time. This aligned access pattern improves spatial and temporal locality in the GPU memory hierarchy, allowing the hardware to better utilize cache capacity and memory bandwidth. Conversely, when requests have heterogeneous prefixes, KV accesses become interleaved across disjoint regions of memory, disrupting locality and reducing effective bandwidth utilization. Another goal which we wish to achieve through our experiments is to evaluate if we can reap locality gains without changing attention kernels.

To validate this hypothesis, we design a series of controlled experiments that systematically vary (i) the amount of prefix sharing within a batch, (ii) the number of distinct prefix groups, and (iii) the batch size and decode length. Across these experiments, we measure decode throughput and memory behaviour to understand how prefix structure affects inference performance. Each experiment concludes with a takeaway summarizing the key insight.

All experiments use the LLama3-8B [3] model. Experiments are performed on an RTX6000 Ada Generation GPU [25], equipped with 48GB of GDDR6 memory and a 96MB L2 cache. We use the vLLM [19] inference engine with a default maximum batch size of 500 unless stated otherwise. We primarily focus on decode-phase throughput, measured in output tokens per second. To isolate decode behaviour, we perform a warm-up run such that all prefix KV caches are resident in GPU memory prior to measurement. This ensures that the per-token compute cost remains constant across experimental settings and that observed differences arise primarily from memory behaviour and batching effects. Later, we also compare end-to-end throughput, which includes the time taken to prefill the queries. For this model, a single 10k-token request generates approximately 1.4GB of KV cache data (across all layers, for both keys and values). Within each layer, KV tensors corresponding to consecutive tokens are stored contiguously in memory. We pick requests from the L-Eval [4] dataset, and we prune the size of each request as required by the experiment. We use the FlashAttention [12] backend throughout all the experiments.

3.2 Locality and Sharing

We begin by examining how *prefix heterogeneity* within a batch affects decode throughput. Prefix heterogeneity refers to the presence of requests with different input prefixes in the same batch. When requests share the same prefix, they access the same KV cache regions during decode; when prefixes differ, the KV accesses correspond to different memory regions.

To quantify the impact of prefix heterogeneity, we design a controlled experiment with two large system prompts, A and B, each of length 10,000 tokens. We form a batch of 500 requests, where each request consists of either prefix A or B, followed by a unique suffix of length 20 tokens (Figure 3). Each request then generates 50 decode tokens. Let $f \in [0, 1]$ denote the fraction of requests using prefix A. When $f = 0$ or $f = 1$, the batch is homogeneous, and all 500 requests share a single prefix. When $0 < f < 1$, two distinct prefix families coexist within the same batch. We do a warmup run before measuring the decode throughput, which ensures that the KV caches corresponding to both the prefix groups are already resident in the GPU memory, and the amount of compute involved is the same across all values of f.

Figure 4 shows decode throughput as a function of f. Throughput is maximized when the batch is homogeneous ($f = 0$ or $f = 1$). However, introducing even a single request from the other prefix family causes a sharp degradation. Specifically, at $f = 0.002$ (one request out of 500), throughput drops by nearly 2×. This demonstrates that decode throughput is extremely sensitive to even minimal prefix heterogeneity within a small batch.

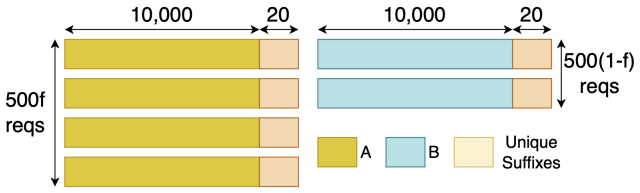


Figure 3. Visualization of Heterogeneous Batches

Do we need to mark fractions such as 0.002 or 0.01 more clearly in this graph?

To better understand this effect, we measure mean DRAM activity, defined as the fraction of cycles during which the DRAM pipe is active. As shown in Figure 5, homogeneous batches exhibit over 50% higher DRAM activity compared to mixed-prefix settings.

A 10k-token prefix corresponds to over 1GB of KV cache data (across all layers). Even with sharing across requests, the active KV working set during decode substantially exceeds the 96MB L2 cache capacity of the GPU, particularly after accounting for model weights. Thus, decode is fundamentally memory-bound, and only a small portion of the KV cache can reside in L2 at any given time.

present max bandwidth numbers here

During each decode step, every request attends to its entire prefix. Because the prefix length (10k tokens) dominates both the decode length (50 tokens) and the query length, memory accesses are largely determined by prefix traversal. In the homogeneous case, all requests sweep through the same KV blocks nearly simultaneously. This produces two beneficial effects: (i) **Spatial Locality:** Sequential traversal of contiguous KV tensors allows the hardware prefetcher to reliably fetch upcoming cache lines from DRAM into L2; (ii) **Temporal Locality:** Since requests access the same KV regions at roughly the same time, a cache line fetched for one request is likely reused by neighbouring requests before eviction. Together, these effects increase L2 hit rates and sustain high DRAM pipeline utilization, resulting in higher throughput. Since we use a query-centric (FlashAttention) backend, most SMs process queries from the dominant prefix group even when $f = 0.002$. Therefore, we attribute the performance gains primarily to improved L2 cache reuse rather than increased L1 cache locality.

In contrast, when even a small fraction of requests uses a different large prefix, the KV access stream becomes interleaved across two disjoint memory regions. This disrupts prefetcher confidence and weakens spatial locality. More importantly, temporal locality collapses; cache lines fetched for prefix A are unlikely to be reused by prefix B requests before eviction. The result is reduced effective bandwidth utilization and significantly lower throughput.

Takeaway 1: Decode throughput is highest when all requests in a batch share the same prefix, allowing their attention operations to access the same KV cache regions at nearly the same time. Even a single request with a different prefix forces the GPU to interleave accesses across multiple KV regions, breaking cache locality and sharply reducing effective memory bandwidth.

The previous experiment raises a natural follow-up question: how does throughput change when requests partially share their prefixes rather than being completely identical or completely different? Our hypothesis is that if decode performance is primarily determined by KV cache locality, then increasing the amount of shared prefix across requests should progressively improve throughput. Larger shared regions should allow more requests to access the same KV blocks within a short time window, strengthening cache reuse and improving effective memory bandwidth utilization.

To test this hypothesis, we isolate the effect of prefix sharing by constructing a batch of 100 requests, each with a total length of 2000 tokens. In this experiment, every request shares a common prefix of length $2000 \times f$ tokens, where $f \in [0, 1]$. The remaining $2000 \times (1 - f)$ tokens are unique to each request, as illustrated in Figure 6. As before, a warm-up run ensures that all the 2000-tokens of each of the 100 requests are already present in the GPU memory, so that we can attribute the trend in throughput solely to the locality present amongst the requests.

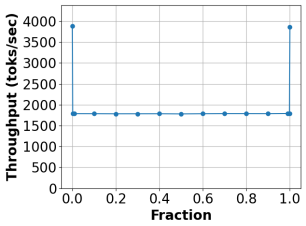


Figure 4. Throughput vs Fraction of A prefix requests

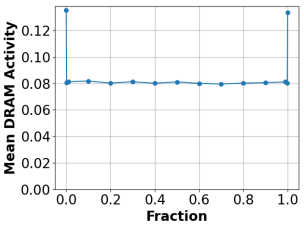


Figure 5. Mean DRAM Activity vs Fraction of A prefix requests

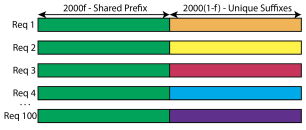


Figure 6. Visualization of $2000 \times f$ tokens shared by requests

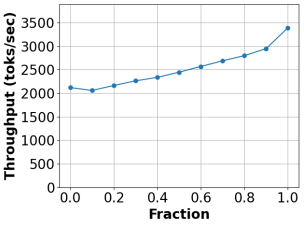


Figure 7. Throughput vs Fraction of Sharing

As shown in Figure 7, there is a smooth increase in throughput with f. When there is no shared prefix ($f = 0$), throughput is at its lowest. As sharing increases, throughput improves consistently. When all requests share the entire prefix ($f = 1$), throughput is more than 60% higher than in the no-sharing case. As f increases, a larger portion of the KV cache becomes common across the batch. Since decode for each token repeatedly sweeps the entire KV cache, the shared region is accessed by multiple requests within a short time window, improving contiguity across memory accesses. Both temporal and spatial locality, therefore, improve with f, resulting in improved throughput.

Takeaway 2: Decode throughput increases smoothly with the amount of shared prefix across requests. Greater sharing enlarges the KV regions accessed jointly by the batch, strengthening cache reuse and improving effective memory bandwidth utilization.

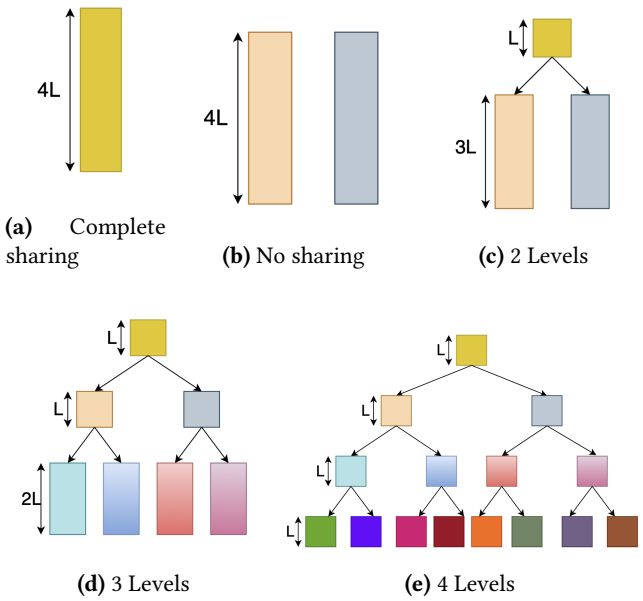
The previous experiment varies only the amount of sharing while assuming that the shared tokens appear at the beginning of the sequence. This raises a natural question: does the position of sharing within a sequence also affect decode throughput? Our hypothesis is that because decode proceeds token by token and all requests traverse their prefixes at the same time, sharing that occurs earlier in the sequence should have a larger impact on locality than sharing that occurs later. Early shared segments allow the batch

to access the same KV regions simultaneously, while later divergence breaks this alignment. To test this hypothesis, we design five sharing patterns: complete sharing, no sharing, and sharing with 2, 3, or 4 levels, as illustrated in Figure 8. In all configurations, the total sequence length of each request is fixed at $4L$, and only the branching structure differs. We measure decode throughput across these five settings for three values of L, as shown in Figure 9.

The complete sharing pattern consistently achieves substantially higher throughput than all others. This behavior is expected: full sharing preserves synchronized traversal of the KV cache throughout the entire sequence, maximizing both temporal and spatial locality.

A more subtle observation is that configurations with 2, 3, and 4 levels exhibit similar throughput across different values of L, despite differing branching patterns. This can be explained by the timing of memory accesses during decode. Requests within a batch issue memory accesses roughly simultaneously. During the initial shared segment of length L, all configurations behave identically; every request accesses the same KV region, fully exploiting locality. Once branching occurs, the batch becomes heterogeneous, and locality degrades. However, as we will show next, throughput is highly sensitive to the presence of heterogeneity but comparatively less sensitive to the exact number of distinct prefix groups once heterogeneity is introduced. Whether the batch ultimately splits into two, four, or eight groups in later segments has only a limited additional impact since synchronized reuse has already been disrupted. Moreover, as expected, no sharing pattern achieves the lowest throughput of all.

Takeaway 3: Because decode-phase memory accesses from all sequences occur roughly together, the amount of sharing at the beginning of the sequence is the primary determinant of locality and throughput.



(a) Complete sharing **(b)** No sharing **(c)** 2 Levels **(d)** 3 Levels **(e)** 4 Levels

Figure 8. Visualization for different sharing patterns

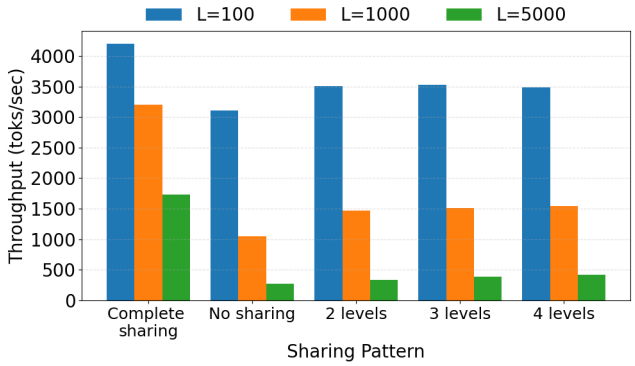


Figure 9. Throughput vs sharing patterns for different sequence lengths

The previous experiments examine how prefix sharing affects decode throughput when the KV caches of all active requests fit within GPU memory. However, in some practical serving workloads, a system may need to handle many distinct prefixes simultaneously. This raises another important question: what happens when the number of distinct prefixes becomes large enough that their KV caches cannot all reside in GPU memory? Our hypothesis is that when the number of active prefixes grows beyond the available GPU memory capacity, the system will be forced to evict KV cache blocks. These evictions require recomputation when the corresponding requests resume their decode phases, leading to reduced throughput and smaller effective batch sizes.

To study this effect, we construct an experiment with N different shared prefixes $P_1, P_2 \ldots P_N$, each of length 5000

tokens. We create batches such that an equal number of requests corresponds to each prefix. Figure 10 shows the resulting throughput and average batch size as N increases. As observed earlier, throughput drops when moving from $N = 1$ to $N = 2$, primarily due to reduced temporal and spatial locality. From $N = 2$ to $N = 20$, throughput remains relatively stable, indicating that locality degradation alone does not worsen performance beyond the initial heterogeneity. However, there is a sharp decline in throughput from $N = 20$ to $N = 50$. This drop occurs because the available GPU memory is no longer sufficient to hold the KV caches for all active prefixes simultaneously. This leads to frequent KV cache evictions and recomputations. The effect is also reflected in the reduced average batch size for $N = 50$ and $N = 100$, as the system cannot admit new requests due to limited GPU memory capacity. This phenomenon has also been explored in prior work [29, 47], which highlights the impact of KV cache capacity constraints on inference throughput.

Takeaway 4: Beyond cache locality, the number of simultaneously active prefixes also affects performance through GPU memory capacity limits. Prefix-aware scheduling can mitigate KV cache evictions and improve throughput.

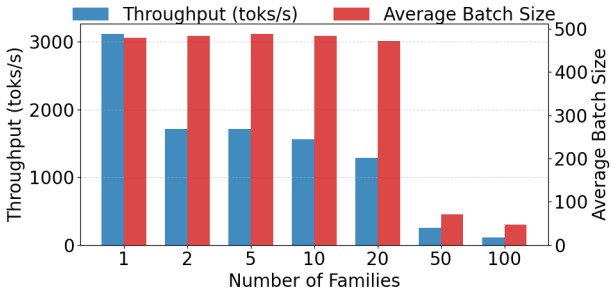


Figure 10. Throughput and Average Batch Size vs Number of Prefix Groups

3.3 Effect of Batch Size on Decode Throughput

The previous experiments show that decode throughput is strongly influenced by prefix structure, which determines how effectively requests can reuse KV cache data. However, prefix sharing is only one factor that affects performance. Another important parameter in LLM serving systems is the batch size, which determines how much parallel work is issued to the GPU during each decode step.

Our hypothesis is that increasing batch size should initially improve throughput by better utilizing GPU compute units, particularly tensor cores. However, once compute resources are saturated, decode performance should become limited by memory bandwidth, since each additional request increases KV cache traffic.

To study this effect, we evaluate decode phase throughput as a function of batch size. In this experiment, all the requests start from one of the 5 large prefixes of length 2000 tokens each. As shown in Figure 11, throughput increases with batch size due to improved tensor core utilization (Figure 25). However, another point worth noting is that beyond a certain batch size, once the GPU's compute units are already highly utilized, the throughput curve largely flattens. The mean SM Occupancy is shown in Figure 38 (currently linked to appendix plot, can remove this).

Takeaway 5: We should aim to increase the batch size to better utilize GPU compute, but only up to the point when the tensor core utilization saturates, after which decode performance is no longer limited by arithmetic throughput but by KV cache movement between HBM and on-chip caches. At that stage, simply adding more requests to the batch does not yield proportional gains because memory becomes the bottleneck.

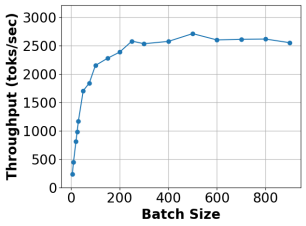


Figure 11. Throughput vs Batch Size

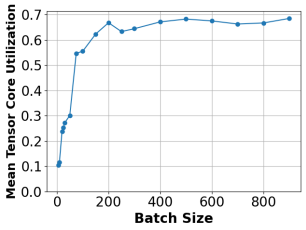


Figure 12. Tensor Core Utilization vs Batch Size

The previous experiment examined how batch size affects decode throughput under a fixed prefix structure. In practice, however, LLM serving workloads involve both prefill and decode phases. These two stages stress different system resources: prefill is primarily compute-intensive, while decode is largely memory-bound due to repeated traversal of the KV cache. Consequently, the benefits of prefix locality observed in earlier experiments may manifest differently when considering end-to-end execution.

Our first hypothesis is that the benefits of prefix locality should become increasingly pronounced as the decode length grows. Prefill is largely a write-heavy workload that constructs the KV cache once, while decode repeatedly reads the entire prefix at every generation step. As the number of decode tokens increases, these repeated KV sweeps amplify the impact of memory locality.

Moreover, there exists a trade-off between large heterogeneous batches and smaller prefix-homogeneous batches. Large heterogeneous batches maximize tensor core utilization, while homogeneous batches preserve aligned traversal of the KV cache during decode.

To evaluate these hypotheses, we measure end-to-end throughput, including both prefill and decode phases. Unlike previous decode-only experiments, we do not perform

a warmup run; thus, prefix computation and KV cache construction are included in the reported throughput. We consider a workload consisting of eight distinct prefix groups, each containing 100 requests (800 total). We evaluate several batching strategies: (i) a single heterogeneous batch of 800 requests spanning all eight families; (ii) two batches of 400 requests covering four families each; (iii) four batches of 200 requests covering two families each; (iv) eight batches of 100 requests, each homogeneous; (v) sixteen batches of 50 requests, each homogeneous; and (vi) thirty-two batches of 25 requests, each homogeneous. We further vary the number of decode tokens to study how the prefill–decode balance influences performance.

Figure 13 shows throughput across these configurations. When the decode length is small, throughput is largely influenced by prefill computation. In this regime, larger heterogeneous batches tend to perform well because they maximize tensor core utilization and arithmetic intensity during the prefix processing stage. As we progressively split the large 1×800 batch into 2×400 and 4×200 configurations, throughput decreases slightly, reflecting reduced compute efficiency from smaller batch sizes. In this setting, the benefits of memory locality are not yet dominant enough to compensate for the loss in arithmetic utilization.

However, as we increase the number of decode tokens, there is a shift in the performance trend. Decode requires each request to repeatedly sweep its entire KV cache. When batches are heterogeneous, these sweeps interleave across disjoint memory regions, weakening spatial and temporal locality. As a result, memory accesses become fragmented and effective bandwidth utilization decreases. In contrast, once we transition to fully homogeneous batches (8×100), throughput increases significantly, even though each batch is smaller than the earlier configurations. This improvement arises because all requests within a batch now share the same prefix, restoring synchronized traversal of the KV cache and strengthening both spatial and temporal locality.

Interestingly, even the 16×50 configuration, which further reduces batch size but remains homogeneous, continues to outperform the single large heterogeneous batch (1×800) when the decode length is sufficiently large. This demonstrates that the locality benefits from prefix alignment can outweigh the loss in tensor core utilization. Only when batches become too small (32×25) does throughput begin to decline again. At that point, reduced arithmetic intensity and lower tensor core occupancy dominate, and the compute under-utilization can no longer be compensated by improved memory alignment.

The impact of decode length further illustrates this trade-off. When the number of decode tokens is small (e.g., 10 tokens), a homogeneous batch of size 50 does not outperform a heterogeneous batch of size 800, since the workload remains largely dominated by prefill computation and compute efficiency plays a larger role. However, when we increase the decode length to 100 tokens, the homogeneous 50-request batch begins to surpass the large heterogeneous batch. This occurs because each additional decode step requires another full sweep of the KV cache, allowing the benefits of aligned memory access patterns to accumulate over time. As decode length increases, execution becomes increasingly memory-bound, and the advantage of prefix-homogeneous batching becomes more pronounced.

Overall, these results highlight a fundamental balance between compute utilization and memory locality. Large heterogeneous batches maximize tensor core usage but fragment KV access patterns. Moderately sized homogeneous batches preserve aligned traversal of the KV cache and can achieve higher effective throughput when decode dominates execution. The optimal operating point lies where compute resources remain well utilized while memory access patterns remain synchronized across requests.

Takeaway 6: Under end-to-end execution, it is often beneficial to process requests in prefix-homogeneous batches, even if this slightly reduces batch size. As the number of decode tokens increases, repeated KV cache sweeps amplify the benefits of memory locality, allowing moderately sized homogeneous batches to outperform larger heterogeneous batches. However, this benefit holds only up to a point: when batches become too small, reduced tensor core utilization and lower arithmetic intensity lead to underutilized compute, causing throughput to decline.

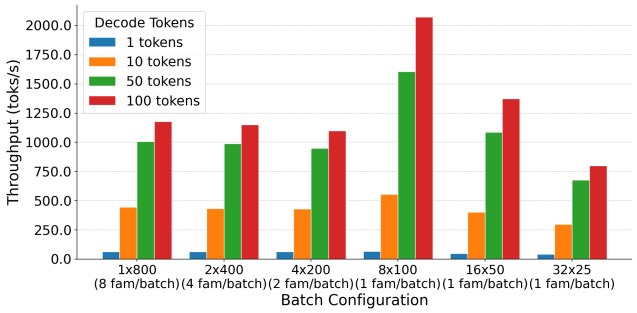


Figure 13. Throughput vs Batch Configurations for varying number of decode tokens

3.4 Overhead of dynamic prefix detection

So far, we have analyzed how the prefix sharing structure and batch size affect the overall inference throughput. Next, we plan to study how the state-of-the-art inference engines identify prefix sharing and what the overall cost involved in doing this is. Although existing inference engines such as SGLang [45] and BatchLLM [47] do not consider making the batch prefix-homogeneous, they still attempt to detect prefix sharing using radix trees to schedule requests in an order that minimizes KV cache recomputations. In SGLang, two radix-tree-aware scheduling policies are proposed. The

Policy	Batch Time (s)	Exec (s)	Throughput	TC Util.
vLLM FCFS	3e-6	229.36	436.25	0.67
vLLM User IDs	0.03	105.43	948.54	0.336
vLLM RadixTree	62.95	139.33	717.35	0.20
SGLANG DFS-W	92.15	177.78	562.61	0.22
SGLANG LPM	46.62	149.27	669.92	0.26

Table 1. Comparison of scheduling policies

first, Longest Prefix Match (LPM), prioritizes requests whose input prefixes match the longest cached prefixes a better definition needed here?. The second, DFS-Weight, performs a depth-first traversal of the radix tree and prioritizes branches that have more active requests. To understand the overhead of these scheduling policies, we implemented a radix-tree-based scheduler inside vLLM and measured its CPU cost. We constructed a workload of 2000 requests, each with a 10k-token prefix, and compared the CPU time spent on scheduling against the total execution time. We also compared these results with an idealized policy in which the user directly provides prefix group identifiers, eliminating the need for any dynamic prefix discovery. describe prefix aware attention here, after reading through, and check if it matches on what we have implemented

Table 1 shows the results. SGLang's LPM and DFS-Weight policies incur 78.24 and 89.06 seconds of CPU time, respectively, on the same order as the GPU computation. Our radix-tree implementation in vLLM reduces this overhead somewhat (57.53 seconds), but the cost remains substantial. In contrast, when prefix group identifiers are provided explicitly, the CPU overhead drops to just 0.03 seconds.

There is a structural reason for this gap: radix trees require token-by-token comparisons during insertion, dynamic node splitting when partial matches occur, and the maintenance of shared paths as requests arrive and complete. Scheduling decisions further require repeated traversals of the tree. With long prompts and high concurrency, these operations accumulate significant constant-factor overhead.

Takeaway 7: Existing approaches introduce significant CPU overhead while detecting prefix sharing, which is at times even comparable to the GPU execution time.

add prefix aware attention tree after reading the paper and implementing their idea

remove vllm radix tree as it has our own optimizations, or we can keep it still?

batchLLM avoids this cost by shifting to offline serving, similar to blendserve.

4 Design and Implementation

As shown in §3, during the decode phase, memory accesses from different requests in a batch are issued almost at the same time. When all requests pass through the same KV cache in the same order, the accesses become nicely sequential, which helps the hardware prefetcher and improves effective DRAM bandwidth utilization. In this setting, as we demonstrated earlier, both temporal and spatial locality naturally emerge. However, this benefit only holds when sharing is universal across the batch. As soon as sharing becomes partial, even slightly, the reuse opportunity drops sharply. Similarly, grouping requests by prefix not only improves locality but also reduces KV cache eviction pressure, thereby preventing expensive recomputation.

Insight 1. The joint objective that emerges is to keep the GPU busy while also preserving shared-prefix locality for as long as possible. A simple way to think about this is through the pair: **(Batch size, Shared prefix tokens common to all requests)**. Increasing batch size improves compute utilization. Increasing the number of shared prefix tokens across all active requests strengthens temporal and spatial locality. The goal, therefore, is not merely to build large batches, but to build large, homogeneous batches.

Challenge 1. At each step of batch formation, where we continuously add requests from the waiting queue into the running scheduler batch, there exists a trade-off between adding a new request to increase GPU utilization and maintaining temporal and spatial locality over a longer shared prefix. This trade-off is sensitive to the interplay between the GPU's compute capability and memory bandwidth, making it difficult to resolve with a fixed heuristic across different kinds of GPUs. In particular, it is unclear when to fall back from a homogeneous batch (all requests sharing a long prefix) to a heterogeneous one (mixed prefixes), especially when insufficient homogeneous requests are available in the queue.

Key Idea 1. We propose learning a reinforcement learning (RL) policy for the scheduler that adaptively resolves this trade-off at runtime, guided by the observed reward.

Insight 2. Beyond scheduling, efficiently identifying which requests share a common prefix is a critical bottleneck in itself. Existing approaches to prefix detection incur significant CPU overhead, causing stalls during inference; particularly as sequence lengths grow to thousands of tokens.

Challenge 2. Designing a data structure that supports fast, low-overhead prefix identification at scale is non-trivial. The structure must handle long sequences efficiently, support dynamic insertion and lookup, and impose minimal CPU cost so as not to bottleneck the inference pipeline.

Key Idea 2. We introduce Chunked Hash Tree, a data structure that detects shared prefixes with substantially lower CPU overhead than existing methods and scales well to longer sequences.

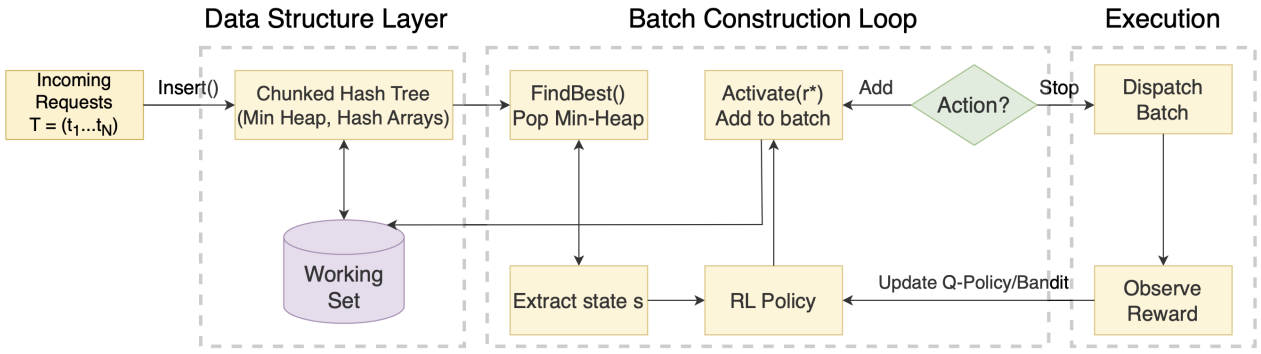


Figure 14. Entire Pipeline

4.1 System Overview

Figure 14 illustrates the overall pipeline of our scheduling system. The goal of the scheduler is to construct batches that both keep the GPU well utilized and preserve spatial-temporal locality for the KV cache. To achieve this, our design combines two main components: (i) a lightweight prefix-tracking data structure called Chunked Hash Tree (CHT) and (ii) a learning-based policy that decides when batch construction should stop.

Request model. We consider an inference server that continuously receives decoding requests. Each request consists of a prompt, which is a sequence of input tokens, followed by autoregressive decoding. At any point in time, a request falls into one of these three states: (i) waiting requests that have not yet been scheduled, (ii) active requests that are currently part of the decoding batch, and (iii) finished requests whose decoding has been completed.

The scheduler operates in repeated decoding steps. During each step, it incrementally constructs a batch by selecting requests from the waiting queue and activating them. Once the batch is finalized, it is executed for one decoding iteration on the GPU.

Prefix-aware request selection. The first challenge is identifying which waiting request should be added next to the batch. Ideally, we want to add requests whose prefixes overlap with those of the already active requests, since this increases temporal and spatial locality. To support this efficiently, we introduce Chunked Hash Tree. Instead of comparing token sequences directly, each request is represented using a compact vector of cumulative prefix hashes computed at fixed token intervals. These hashes allow the scheduler to quickly detect shared prefixes across requests. The CHT maintains a working set that represents the prefixes currently referenced by active requests and tracks how much prefix overlap each waiting request has with this working set. Using this information, the scheduler can efficiently identify the waiting request that maximizes our joint objective function.

Batch construction loop. Using the CHT, the scheduler repeatedly retrieves the best candidate request from the waiting queue (the one with the highest prefix overlap) and considers adding it to the current batch. Each candidate provides useful statistics, such as the current batch size, the shared-prefix length across all active requests before and after the addition, and the number of waiting requests that could benefit from the same prefix. At this point, the scheduler must decide whether to add the request or stop building the batch. Adding more requests improves GPU utilization but may reduce prefix locality if the shared prefix becomes smaller.

Learning when to stop. We formulate this decision as a learning problem. The scheduler observes a state that summarizes the current batch size and prefix-sharing statistics, and a reinforcement learning policy chooses between two actions: Add the candidate request to the batch or Stop and dispatch the batch for execution. After the batch is executed, the system observes the resulting performance (e.g., throughput) and uses it as feedback to update the policy. Over time, the policy learns how to balance GPU utilization and prefix locality for different workload conditions.

End-to-end flow. Putting everything together, the pipeline works as follows. Incoming requests are first inserted into the CHT, where their prefix hashes are computed and indexed. During batch construction, the scheduler repeatedly calls FindBest to retrieve the best candidate request based on prefix overlap. The reinforcement learning policy then decides whether to activate the request and add it to the batch or to stop. Once the batch is finalized, it is dispatched for execution on the GPU. The observed throughput is then used to update the policy for future scheduling decisions.

The following subsections describe the Chunked Hash Tree data structure in detail and then present the batching and reinforcement learning policies built on top of it.

4.2 Chunked Hash Tree

a better name, without a tree, for uniqueness. Do we need to make it a separate section?

starting with a brief overview

need to mention that implemented in C++ (can do so in the section integration with vllm)

Motivation and Design Evolution. The scheduler repeatedly needs to answer a simple question: among all waiting requests, which one shares the largest prefix with the currently active batch? Requests that share longer prefixes will lead to higher temporal and spatial locality. Since scheduling decisions occur frequently and the waiting queue may contain thousands of requests with long prompts, detecting prefix overlap must be extremely fast and impose minimal CPU overhead.

Modern inference systems already exploit prefix sharing for KV-cache reuse. As shown in §2, vLLM [19] uses Merkle Trees, while SGLang [45] and BatchLLM [47] use radix trees to detect prefix sharing. Our scheduler, however, has a different objective: we do not need to store or manage KV-cache blocks; instead, we only need a lightweight way to estimate how much prefix overlap exists between requests so that we can select the best request to add to the batch. While a radix tree precisely captures prefix sharing, it incurs significant overhead; every insertion may trigger node splitting, and finding the best matching waiting request requires a DFS traversal that is $O(W)$ in the worst case, where W is the number of waiting requests. For a scheduler making frequent decisions over thousands of long-prompt requests, that is quite expensive. Similarly, Merkle-style hash chains introduce unnecessary sequential dependencies during hash computation and expose more structure than the scheduler actually requires.

We instead want a structure centered around a working set: the set of prefix chunks currently held by active requests; so that we can directly ask: "which waiting request has the most overlap with the already running batch?" This reframes the problem from tree traversal to set intersection. To that end, rather than maintaining a strict hash chain, we divide the token sequence into fixed-size chunks and compute prefix hashes at chunk boundaries. An additional advantage of this design is that hash computation can be naturally batched and parallelized. Because chunks are processed independently, their hash contributions can be computed separately and later combined to obtain the cumulative prefix hashes. This pattern is similar in spirit to techniques such as online softmax, where partial statistics are computed over blocks and then merged to produce the final result. In contrast, Merkle-style hash chains require sequential computation because each hash depends on the previous one.

The simplest realization is to represent each request as a set of hashed prefix chunks, maintain the working set explicitly, and, on each scheduling decision, scan all waiting requests and count how many of their chunks appear in the working set. The request with the highest overlap wins. This is correct and easy to implement, but finding the best request is $O(W \cdot C)$ — linear in the number of waiting requests times chunks per request, since we rescan everything on every call.

We can avoid rescanning by maintaining a missing count per waiting request: the number of its chunks that are absent from the working set. This count only needs to change when the working set itself changes, i.e., upon activation or completion of a request. Instead of recomputing overlaps from scratch, we propagate small delta updates. Finding the best request then becomes a linear scan over waiting requests to find the minimum missing count — $O(W)$.

To eliminate the linear scan, we maintain a min-heap ordered by missing count. When a chunk enters or leaves the working set, we update the missing counts of affected waiting requests and push new entries to the heap. However, in-place heap updates are expensive, so we use lazy deletion: push a new entry with the updated count, and skip stale entries (where the heap's recorded count differs from the current missing cache) when popping. This reduces the complexity to $O(\log W)$ amortized, while keeping activation and completion costs proportional only to the number of affected waiting requests per chunk—typically far smaller than W.

The resulting structure, which we call the Chunked Hash Tree (CHT), is a scheduling data structure that maximizes KV-cache prefix reuse across concurrent inference requests. We first describe how requests are represented as hash vectors, then introduce the data structures that track the scheduler state, and finally walk through each operation in turn.

Hash Computation. Before a request can be inserted into the scheduler, its token sequence must be converted into a compact representation that captures the prefix-sharing structure. As shown in Figure 16, given a request with a token sequence $T = (t_1, t_2, \ldots, t_N)$, we partition T into non-overlapping chunks of size K (e.g., $K = 50$). For each chunk boundary $\ell = 1, \ldots C$ where $C = \lceil N/K \rceil$, we compute a cumulative prefix hash at each boundary

$$h_\ell = \text{hash}(t_1, t_2, \ldots, t_{\min(\ell K, N)}), \quad \ell = 1, \ldots, C.$$

Because the hash is cumulative, two requests share the same h_ℓ if and only if their first ℓK tokens are identical, which is precisely the condition under which their KV-cache prefixes can be shared. Each request r is therefore represented as an ordered hash vector $H_r = (h_1^r, h_2^r, \ldots, h_C^r)$.

Algorithm 1 details the hash computation. We use an incremental xxHash-64 state so that each token is processed exactly once, making the total cost $O(N)$ regardless of the number of chunked boundaries.

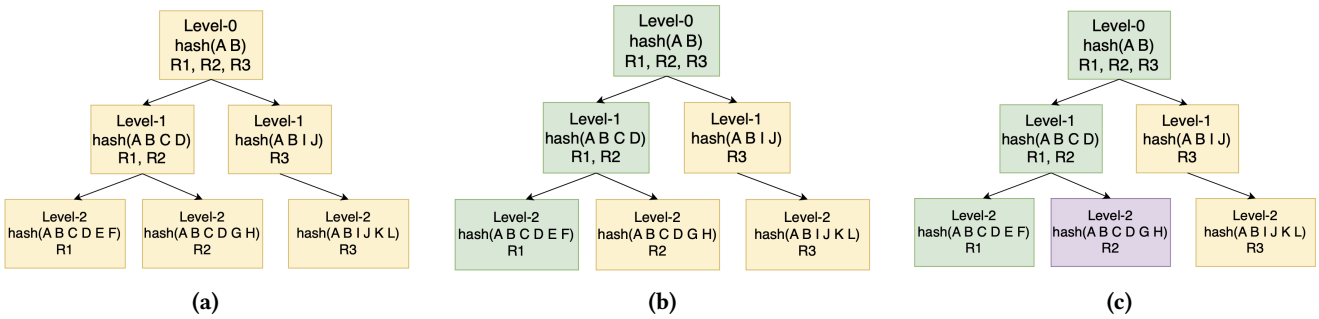


Figure 15. An example of chunked hash tree workflow which shows insertion, activation and best request selection. Consider 3 requests R_1, R_2, R_3 each with 6 tokens. With a chunk size of 2, each request will get divided into 3 levels, as shown in 15a. Next, we add R_1 to the active decoding batch, this makes the shared prefix chain include all its nodes, which are marked as green in 15b. Now, we wish to find the next candidate request which can be added to the running batch. We have two options R_2 and R_3. Since, R_2 has the max overlap with the current shared prefix chain, we prefer it over R_3, as shown in 15c.

Algorithm 1 ComputeHashes(T, K)

Require: Token sequence $T = (t_1, \ldots, t_N)$; chunk size K
Ensure: Cumulative prefix-hash vector $H = (h_1, \ldots, h_C)$

```
H ← [];   C ← ⌈N/K⌉
Initialise incremental xxHash-64 state s
for i ← 1 to N do
    Feed t_i into s
    if i mod K = 0 or i = N then
        H.Append(Digest(s))
    end if
end for
return H
```

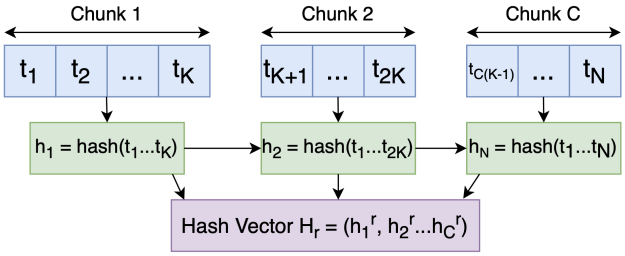


Figure 16. Hash Computation in Chunked Hash Tree

Data Structures. The CHT maintains the following components that together capture the full scheduler state:

Hash vectors. A map from request ID to its cumulative prefix-hash vector $H_r = (h_1^r, \ldots, h_C^r)$ is stored for every active and waiting request.

Reference counts. `ref` maps each h to the number of currently active requests that contain that hash in their hash vectors.

Working set. `working_set` is the set of hashes that belong to at least one active request. A hash is in the working set if and only if its reference count is greater than zero.

Missing cache. `miss` maps each waiting request r to an integer m_r, defined as the number of hashes in H_r that are not currently in the working set. Intuitively, m_r counts how many of r's prefix chunks would need to be freshly computed (i.e., are not already cached by the active batch). A request with $m_r = 0$ has its entire prefix covered by the current active batch and is therefore the most preferable candidate to admit next.

Waiting heap. `waiting_heap` is a min-heap of (m_r, r) pairs, ordered by missing count m_r. It enables $O(\log W)$ retrieval of the waiting request with the smallest missing count, where W is the number of waiting requests.

Reverse index. `waiting` maps each hash h to the set of waiting requests whose hash vector contains h, enabling $O(1)$ updates to `miss` when the working set changes.

Shared prefix chain. `shared_prefix_chain` is a vector $P = (p_1, \ldots, p_L)$ recording the longest common hash prefix shared by all currently active requests. P is maintained incrementally rather than recomputed from scratch for every event, keeping the amortized cost low.

Request Insertion (Algorithm 2). Insert is called the moment a new request arrives at the scheduler. It computes H_r via ComputeHashes, stores the hash vector, and for each hash $h \in H_r$, registers r in the reverse index $waiting[h]$. Crucially, it also computes the initial missing count m_r: the number of hashes in H_r not yet present in the working set. This count is stored in $miss[r]$ and the pair (m_r, r) is pushed onto the waiting heap. No reference counts are modified at this stage; that happens only when the request is activated.

Algorithm 2 INSERT(r, T)

Require: Request r; token sequence T; working set *working_set*; reverse index *waiting*; min-heap *waiting_heap*
Ensure: r registered and ready for scheduling

$H_r \leftarrow$ COMPUTEHASHES(T, K)
request_hashes$[r] \leftarrow H_r$
$m_r \leftarrow 0$
waiting_requests $\leftarrow$ *waiting_requests* $\cup \{r\}$
for $\ell \leftarrow 1$ **to** $|H_r|$ **do**
  if $h_\ell^r \notin$ *working_set* **then**
    $m_r \leftarrow m_r + 1$
  end if
  waiting$[h_\ell^r] \leftarrow$ *waiting*$[h_\ell^r] \cup \{r\}$
end for
miss$[r] \leftarrow m_r$
waiting_heap.PUSH$(m_r,\ r)$

Activation (Algorithm 3). ACTIVATE is called when the scheduler moves a request from the waiting queue into the active decoding batch. For each hash h in H_r, if h transitions from absent to present in the working set (i.e., its reference count goes from 0 to 1), then h is added to the working set and all waiting requests that contain h have their missing count decremented by one; updated pairs are pushed onto the heap. This eager propagation keeps *miss* accurate for all waiting requests without requiring a full rescan. The shared prefix chain is then updated incrementally via UPDATECHAINONACTIVATE (Algorithm 4).

Algorithm 3 ACTIVATE(r)

Require: Request r with hash vector H_r; ref counts ref; working set *working_set*; reverse index *waiting*; missing cache *miss*; min-heap *waiting_heap*
Ensure: Updated ref, *working_set*, *miss*; updated shared prefix chain

waiting_requests $\leftarrow$ *waiting_requests* $\setminus \{r\}$;
active $\leftarrow$ *active* $\cup \{r\}$
added $\leftarrow 0$
for $\ell \leftarrow 1$ **to** $|H_r|$ **do**
  if ref$[h_\ell^r] = 0$ **then** ▷ h_ℓ^r newly enters working set
    working_set $\leftarrow$ *working_set* $\cup \{h_\ell^r\}$
    for $w \in$ *waiting*$[h_\ell^r]$ **do**
      if $w \notin$ *active* **and** $w \in$ *miss* **then**
        miss$[w] \leftarrow$ *miss*$[w] - 1$
        waiting_heap.PUSH(*miss*$[w],\ w$)
      end if
    end for
    added $\leftarrow$ *added* $+ 1$
  end if
  ref$[h_\ell^r] \leftarrow$ ref$[h_\ell^r] + 1$
end for
UPDATECHAINONACTIVATE(r)
return (*added*, $|H_r|$)

Updating the shared prefix chain on activation (Algorithm 4). It is called at the end of every ACTIVATE after r has been added to the active set. It keeps the shared prefix chain consistent without recomputing it from scratch. Because adding a new request can only shorten the common prefix (never lengthen it), we must find the longest prefix of the current chain P that H_r also satisfies.

As described before, each hash h_ℓ is produced by a cumulative (prefix) hasher: h_ℓ is the digest of all tokens from position 1 through $\ell \cdot C$, where C is the chunk size. Consequently, if two hash vectors agree at level ℓ, they necessarily agree at every level $1, \ldots, \ell - 1$ as well, because those earlier digests are sub-sequences of the same cumulative stream. Formally, we define the predicate $\mathcal{M}(\ell) = \left[h_\ell^r = p_\ell\right]$. The cumulative property guarantees that $\mathcal{M}$ is monotone decreasing on the index set $\{1, \ldots, \min(L, |H_r|)\}$: once $\mathcal{M}(\ell)$ is false for some ℓ^*, it is false for all $\ell > \ell^*$. This monotone structure is exactly the condition required for binary search to be correct: we seek the largest k such that $\mathcal{M}(k)$ is true, and binary search finds it in $O(\log \min(L, |H_r|))$ comparisons rather than the $O(\min(L, |H_r|))$ required by a linear scan.

Algorithm 4 UpdateChainOnActivate(r)

Require: Request r with hash vector H_r; current shared chain P of length L; active set *active*
Ensure: P reflects the longest common prefix of all active requests

if $|active| = 1$ **then** ▷ r is the only active request
  $P \leftarrow H_r$; $L \leftarrow |H_r|$
  return
end if
$L \leftarrow \text{BinarySearch}(H_r, P, \min(L, |H_r|))$ ▷ largest k s.t. $h^r_k = p_k$
Truncate P to its first L entries

Completion (Algorithm 5). Finish is called when the model has generated the final token for request r, and the request is removed from the active batch. For each hash h in H_r, its reference count is decremented; if it reaches zero, h is evicted from the working set, and all waiting requests containing h have their missing count incremented by one, with updated pairs pushed onto the heap. The shared prefix chain is then updated via UpdateChainOnFinish, since removing a request can only lengthen the common prefix.

Algorithm 5 Finish(r)

Require: Request r with hash vector H_r; ref counts ref; working set *working_set*; reverse index *waiting*; missing cache *miss*; min-heap *waiting_heap*
Ensure: Updated ref, *working_set*, *miss*; reverse index cleaned; updated shared prefix chain

$active \leftarrow active \setminus \{r\}$
$evicted \leftarrow 0$
for $\ell \leftarrow 1$ **to** $|H_r|$ **do**
  $\text{ref}[h^r_\ell] \leftarrow \text{ref}[h^r_\ell] - 1$
  if $\text{ref}[h^r_\ell] = 0$ **then**
    $working_set \leftarrow working_set \setminus \{h^r_\ell\}$
    for $w \in waiting[h^r_\ell]$ **do**
      if $w \notin active$ **and** $w \in miss$ **then**
        $miss[w] \leftarrow miss[w] + 1$
        $waiting_heap$.Push($miss[w]$, w)
      end if
    end for
    $evicted \leftarrow evicted + 1$
  end if
end for
UpdateChainOnFinish(r)
Remove(r) ▷ cleans *waiting* index and *request_hashes*
return ($evicted$, $|H_r|$)

Updating the shared prefix chain on completion (Algorithm 6). It is called inside Finish, before r's hash vector is erased, after r has been removed from the active set. Removing a request can only extend the common prefix. We therefore try to grow the chain level by level beyond its current length by checking whether all remaining active requests agree on each successive hash. Two special cases are handled up front: if no requests remain, the chain is cleared; if exactly one request remains, the chain is set to its full hash vector.

Algorithm 6 UpdateChainOnFinish(r)

Require: Active set *active* (already excludes r); current shared chain P of length L; hash vectors *request_hashes*
Ensure: P extended as far as all remaining active requests agree

if $active = \emptyset$ **then**
  $P \leftarrow []$; $L \leftarrow 0$; **return**
end if
if $|active| = 1$ **then**
  RecomputeChain(); **return** ▷ sets P to the sole remaining request's full hash vector
end if
$M \leftarrow \min_{q \in active} |H_q|$ ▷ maximum possible extension
for $\ell \leftarrow L + 1$ **to** M **do**
  $h^* \leftarrow h^{q_0}_\ell$ for an arbitrary $q_0 \in active$
  if $\forall\, q \in active : h^q_\ell = h^*$ **then**
    P.Append(h^*); $L \leftarrow L + 1$
  else
    break
  end if
end for

Finding the Best Request (Algorithm 7). FindBest is called by the batch-construction loop whenever the scheduler needs to select the next candidate request to admit. The key insight is that the best candidate is the waiting request whose prefix chunks are most covered by the current active batch, i.e., the one with the smallest missing count m_r. The algorithm extracts the minimum-missing-count entry from the waiting heap, skipping stale entries (those whose recorded missing count no longer matches *miss* due to lazy deletion). Three auxiliary statistics are then computed to guide the caller's admission decision: (i) *chunks_before*, the shared prefix chain length before the candidate is admitted; (ii) *chunks_after*, the chain length that would result after admitting the candidate, computed by comparing H_{r^*} with P; and (iii) *peers*, the number of other waiting requests sharing the prefix up to level *chunks_after*, indicating the future benefit that the admission would unlock.

The result is cached and reused on repeated calls until the working set or waiting queue changes, avoiding redundant heap traversals.

Algorithm 7 FindBest()

Require: Min-heap *waiting_heap* of (m_r, r) pairs; missing cache *miss*; active set *active*; shared chain P of length L
Ensure: $(r^*, \textit{chunks_before}, \textit{chunks_after}, \textit{peers})$ or $\perp$ if the waiting queue is empty

if *waiting_requests* $= \emptyset$ **then return** $\perp$
end if
if *cache_valid* **then return** *cached_result*
end if
while *waiting_heap* $\neq \emptyset$ **do**
  $(m, r) \leftarrow$ *waiting_heap*.Pop()
  if $r \in$ *active* **then continue** ▷ already activated
  end if
  if $r \notin$ *miss* **then continue** ▷ request no longer exists
  end if
  if *miss*$[r] \neq m$ **then continue** ▷ stale heap entry
  end if
  $r^* \leftarrow r$
  chunks_before $\leftarrow L$
  chunks_after $\leftarrow$ ChainLengthWith(r^*) ▷ compare H_{r^*} with P
  peers $\leftarrow$ WaitersAtLevel(*chunks_after*)
  waiting_heap.Push(m, r^*) ▷ restore entry for future calls
  Cache and **return** $(r^*, \textit{chunks_before}, \textit{chunks_after}, \textit{peers})$
end while
return $\perp$

Figure 15 shows a worked example of insertion, activation, and best-request selection. Figure 17 shows how the shared prefix chain shrinks and extends upon activation and completion of requests.

Optimizations.

Result caching. FindBest() may be called repeatedly within a single scheduling round without any intervening state change. We cache its last result—a four-tuple $(r^*, \textit{chunks_before}, \textit{chunks_after}, \textit{peers})$ and return it immediately on subsequent calls until a mutation invalidates the flag.

Lazy heap updates. The waiting heap uses lazy deletion: when a request's missing count changes, we push a new (m_r, r) entry rather than updating the existing one in place. Stale entries (where the recorded m differs from *miss*$[r]$) are silently skipped during FindBest. This avoids the $O(W)$ cost of a linear heap search and keeps activation and completion at $O(C \cdot W_h \cdot \log W)$ total, where W_h is the average number of waiting requests per hash.

Incremental shared prefix chain maintenance. Recomputing the shared prefix chain from scratch after every activate or finish event would cost $O(A \cdot C)$, where A is the active-set size. Instead, we exploit the fact that activation can only shrink the chain and completion can only grow it. On activation, we truncate P at the first mismatch with the new request's hash vector in $O(L)$; on completion, we attempt a forward extension beyond the current chain length, stopping at the first disagreement, again in $O(L + C_{\max})$.

Additionally, all major hash tables and vectors pre-allocate capacity at construction time, eliminating rehash latency and reallocation during high-throughput operation.

Complexity Analysis. Table 2 summarizes the time complexity of each operation. T is the average number of tokens per request, C is the average number of chunks per request, W is the number of waiting requests, and W_h is the average number of waiting requests sharing a given hash (typically $W_h \ll W$).

Table 2. Time complexity of CHT scheduler operations.

Operation	Complexity
Insert request	$O(T + C \log W)$
Activate request	$O(C \cdot W_h \cdot \log W)$
Update chain on activate	$O(L)$
Finish request	$O(C \cdot W_h \cdot \log W)$
Update chain on finish	$O(L + C_{\max})$
Find best request	$O(\log W)$ (cached: $O(1)$)
Remove request	$O(C)$

Here $L \leq C$ is the current shared chain length, and $C_{\max}$ is the maximum number of chunks in any active request. Because result caching eliminates redundant heap traversals between working-set changes, the dominant cost in practice is the $O(C \cdot W_k \cdot \log W)$ propagation step in activate and finish, which updates the missing counts of all affected waiting requests.

4.3 Batching and Reinforcement Learning

any visualizations for these RL policies needed?

So far, the CHT provides us with an efficient way to find the best next request to add to the batch, the one that maximizes prefix reuse. But it does not tell us when to stop. Adding more requests always increases GPU utilization, but at some point, the shared prefix starts to shrink, hurting locality. This section describes how we frame this stop/continue decision as a learning problem and present three policies of increasing complexity: a hand-crafted heuristic that encodes domain intuition, a contextual bandit that learns from experience without temporal structure, and a Q-learning agent that reasons about long-term throughput.

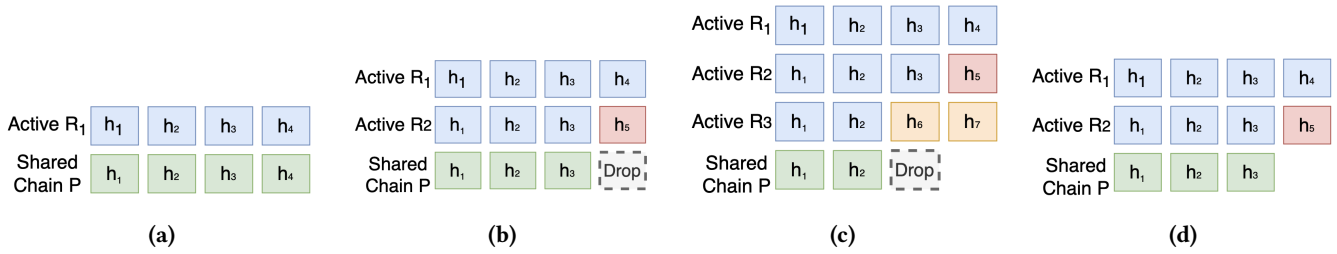


Figure 17. An example of shared prefix chain workflow. We initially have just a single active request R_1 with four prefix hash chunks, so the shared prefix chain is also the same as its hash vector, as shown in figure 17a. We then activate R_2, which does not have the last level common with R_1, and thus the chain size reduces by one 17b. Similarly, as shown in 17c, activation of another request R_3 leads to further reduction in the chain size. Now, R_3 finishes its execution first, so we remove it from the set of active requests, and therefore this gives an opportunity to extend the chain 17d.

As highlighted in §4.1, the scheduler incrementally builds a decoding batch by repeatedly selecting the lowest-cost waiting request via the CHT and deciding whether to add it to the current batch or dispatch the batch as-is. This stop/continue decision is the crux of the batching problem.

State, Actions, and Reward. At each step of batch formation, the scheduler observes a state

$$s = (\textit{batch_size},\ \textit{chunks_before},\ \textit{chunks_after},\ \textit{requests_waiting})$$

and takes a binary action $a \in \{\textsc{Add}, \textsc{Stop}\}$. Here, *batch_size* denotes the number of requests that are already in the batch, which is a proxy for current GPU utilization. *chunks_before* is the shared-prefix chain length before adding the candidate request; *chunks_after* is the chain length after hypothetically adding it (obtained from FindBest); and *requests_waiting* is the number of waiting requests that share the prefix up to level *chunks_after*, indicating whether the locality cost of this addition will be compensated by future activations. Together, these four values capture both the current GPU utilization (batch size) and the cost of the proposed addition to prefix locality, summarized by the chunk loss $\Delta = \textit{chunks_before} - \textit{chunks_after}$).

Both learning policies share the same reward signal: the decode throughput of the dispatched batch,

$$\mathcal{R} = \text{throughput}_{\text{decode}}$$

Maximizing this reward simultaneously encourages large batches (better GPU utilization) and short decode times (better prefix locality). Both policies also operate over the same discretized state space: *batch_size* and *requests_waiting* are mapped to exponential bins, while the chunk loss Δ is mapped to a coarser set of thresholds. The discretized state is a triple $(\hat{b}, \hat{\Delta}, \hat{w})$, where each hat denotes the bin index. The resulting table is small enough to store in a hash map and converges quickly, even under moderate traffic.

Heuristic Policy. Before introducing any learning, we establish a baseline that encodes domain intuition directly as rules. The heuristic is designed around two competing goals: maximizing GPU utilization by filling the batch as much as possible and preserving prefix locality by avoiding candidates that cause large drops in the shared prefix chain. The heuristic always adds a candidate that does not hurt the shared chain at all—a free addition. When the batch is still very small and GPU utilization is clearly low, it tolerates a slightly larger locality cost to avoid dispatching an underutilized batch. In the general case, it accepts a candidate only if the chunk loss is within a small tolerance. Finally, when many waiting requests share the same prefix level, it allows a marginally higher locality cost since the short-term penalty is likely to be offset by the homogeneous additions that follow. If none of these conditions are met, the heuristic stops and dispatches the current batch.

This policy requires no training and converges immediately, but its thresholds are fixed and cannot adapt to shifts in workload distribution.

Contextual Bandit. To move beyond hand-tuned thresholds, we implement a contextual bandit using Upper Confidence Bound (UCB) selection [6]. The bandit treats each batch-construction call as an independent decision: given the discretized state $\hat{s} = (\hat{b}, \hat{\Delta}, \hat{w})$, it selects the action that maximizes

$$\text{UCB}(\hat{s}, a) = \frac{\mu(\hat{s}, a)}{n(\hat{s}, a)} + c\sqrt{\frac{\ln N}{n(\hat{s}, a)}}$$

where $\mu(\hat{s}, a)$ is the cumulative reward for taking action a in state $\hat{s}$, $n(\hat{s}, a)$ is the number of times that pair has been selected, N is the total number of steps, and c is the exploration coefficient. The first term favors actions that have historically yielded high throughput; the second term ensures that under-explored actions are tried sufficiently often.

Because the bandit discards temporal structure entirely, it does not reason about how today's batching decision affects the state of the queue for the next batch, it can miss opportunities where a short-term locality sacrifice leads to a

better-amortized sequence of future batches. This motivates the full reinforcement learning approach.

Q-Learning Policy. To capture the temporal dependencies that the bandit ignores, we model batch construction as an episodic reinforcement learning problem. A new episode begins each time the scheduler starts forming a fresh batch and terminates when it issues Stop or the waiting queue is exhausted. The Q-table [35] stores a value $Q(s, a)$ for every (discretized state, action) pair and is updated upon each batch completion via the standard Bellman update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

where r is the observed throughput reward, s' is the state at the start of the next batch formation, α is the learning rate, and γ is the discount factor. The discount factor encodes that the agent values near-term throughput slightly more than long-term gains, which matches the real-time nature of an inference server. During training, ε starts at 1.0 (fully random) and decays linearly to $\varepsilon_{\min}$ over a configurable number of steps, after which the policy is nearly deterministic.

The BuildBatch Procedure. Algorithm 8 shows how all three policies fit into the same batch-construction loop. At each iteration, FindBest (Algorithm 7) is called to retrieve the lowest-cost candidate from the CHT along with its chain statistics c_{before}, c_{after}, and the number of waiting requests w that share the same prefix level. These four values are assembled into the state s and passed to whichever policy π is active—heuristic, bandit, or Q-learning agent—which returns either Add or Stop. If Add, the candidate is activated and appended to the batch; if Stop, the loop exits and the batch is dispatched immediately. Once the batch executes, the observed decode throughput serves as the reward signal: the bandit updates its count-and-mean tables, while the Q-learning agent applies the Bellman update above. The heuristic requires no update.

Algorithm 8 BuildBatch()

Require: Waiting queue Q; policy π (heuristic, bandit, or Q-learning); discount γ; learning rate α
Ensure: Batch B ready for one decode iteration

```
B ← ∅
while Q ≠ ∅ do
    (r*, c_before, c_after, w) ← FindBest()
    if r* = ⊥ then break
    end if
    s ← (|B|, c_before, c_after, w)
    a ← π(s)  ▷ heuristic rule / UCB look-up / ε-greedy Q-table
    if a = Stop then break
    end if
    Activate(r*);   B ← B ∪ {r*}
end while
throughput_decode ← ExecuteBatch(B)
Update π with observed reward throughput_decode  ▷ no-op for heuristic
return B
```

4.4 Integration with vLLM and SGLang

We integrate our scheduler as a lightweight layer on top of both vLLM's [19] and SGLang's [45] existing request management pipeline. The Chunked Hash Tree (implemented in C++), shared prefix chain tracking, and reinforcement learning policies operate entirely at the scheduler level.

5 Evaluation

We evaluate Feather on a variety of popular models, GPU configurations, and two datasets. Our evaluation seeks to answer the following questions:

1. How does our scheduler's performance scale across diverse workloads, varying numbers of prefix groups, decode tokens, and input sequence lengths?
2. What performance improvement does Feather achieve for both online and offline serving under real-world workloads?
3. What is the contribution of each component of Feather to the overall performance gain?
4. How much throughput degradation is caused by the replacement of some components with the ideas presented in existing work?
5. What is the impact of Feather's overhead?

5.1 Experimental Setup

Model and server configurations. We use Qwen [7] models with 2B parameters, LLama3 [3] with 8B parameters, and LongChat [22] with 13B parameters. For most of our experiments, we use the RTX 6000 Ada Generation GPU [25] with 48 GB of HBM and a cache size of 96 MB for L2. The maximum batch size is set to 500 for all these experiments.

present results on another GPU? A100 for PAT? Any other GPU needed too?

Workloads: Similar to prior work, the arrival pattern of requests is generated by a Poisson process. The input lengths and output lengths of requests are sampled from the following real-world datasets.

L-Eval [4]: It contains human-labeled query-response pairs on diverse tasks, such as summarization and question answering. The range of sequence length in this dataset is 2.7K - 210.5K tokens.

LongBench [8] encompasses a curated subsample of elongated data, ranging from 4K to 10K context lengths, excerpts from 21 datasets across 6 categories, including tasks such as multi-document question answering, summarization, and code completion.

We consider vLLM [19] (FCFS), SGLang [45] (all three policies - FCFS, LPM, DFS-W), and DynamicBatching [30] as our baselines because they represent the state-of-the-art for LLM inference.

Key Metrics. Throughput (toks/s) measures how many output tokens the system can generate per second and indicates its overall serving capacity. Higher throughput indicates better utilization of GPU compute and memory resources. Time Between Tokens (TBT) is the average time gap between consecutive generated tokens during decoding. It reflects how smoothly and responsively the model produces output, which is especially important for interactive settings. Along with these two metrics, we also measure the average batch size during the inference process.

5.2 Main Results

- Separate SGLANG and vLLM if needed?
- Throughput, Average Batch Size, TBT across req rate?
- Throughput, Average Batch Size, TBT across number of prefix groups?
- Throughput, Average Batch Size, TBT across varying decode lengths?
- Throughput, Average Batch Size, TBT across varying context length (number of tokens shared by system prompt)?
- Throughput, Average Batch Size, TBT across different models?
- Throughput, Average Batch Size, TBT across different radix tree modes?

5.3 Micro-benchmarks

- Compare the CPU overhead of the chunked hash tree, basically by adding a row to the previous table? Add the cycles taken by individual functions of our chunked hash tree.
- How do the state and action flow for these two RL policies? How does the batch size vary across iterations?
- Also present DRAM activity numbers here?

5.4 Ablation Study

- Replace chunked hash tree by token level radix tree, and compare both CPU overhead and drop in overall throughput.
- Remove RL policy, do not stop the batch, and compare the throughput in low request rate settings, where larger batch sizes will reduce the homogeneity.

6 Conclusion

References

[1] 0xeb. 2024. The Big Prompt Library: A Collection of Prompts, System Prompts and LLM Instructions. https://github.com/0xeb/TheBigPromptLibrary. GitHub repository.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. arXiv:2403.02310 [cs.LG] https://arxiv.org/abs/2403.02310

[3] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[4] Chenxin An, Shansan Gong, Ming Zhong, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2023. L-Eval: Instituting Standardized Evaluation for Long Context Language Models. arXiv:2307.11088 [cs.CL]

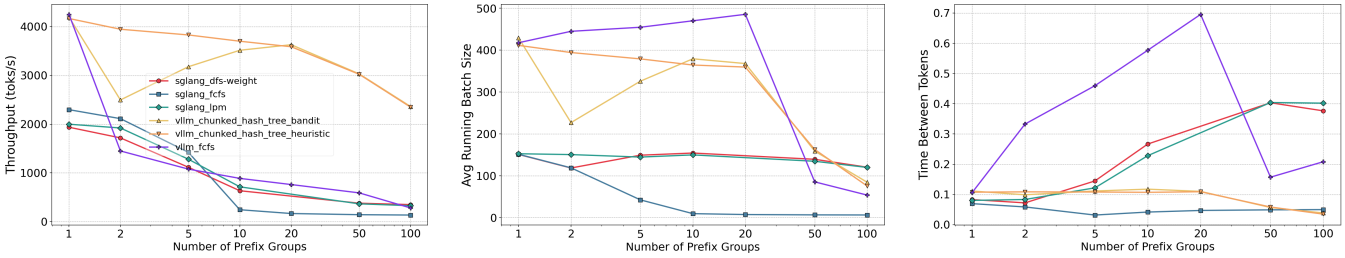
[5] Anthropic. 2024. Claude. https://claude.ai.

[6] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning 47, 2-3 (2002), 235–256. doi:10.1023/A:1013689704352

[7] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv preprint arXiv:2309.16609 (2023).

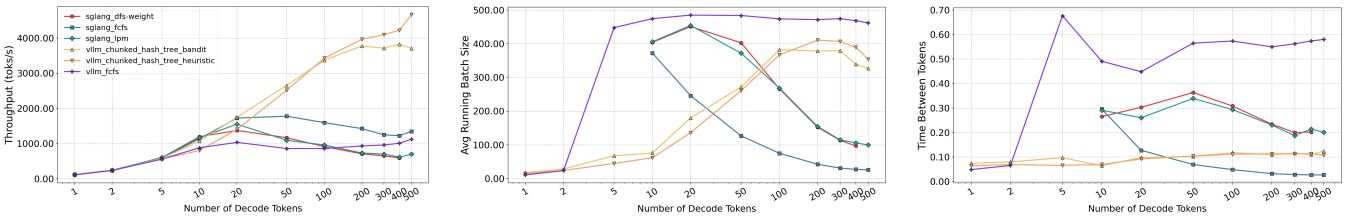
[8] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. arXiv preprint arXiv:2412.15204 (2024).

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165



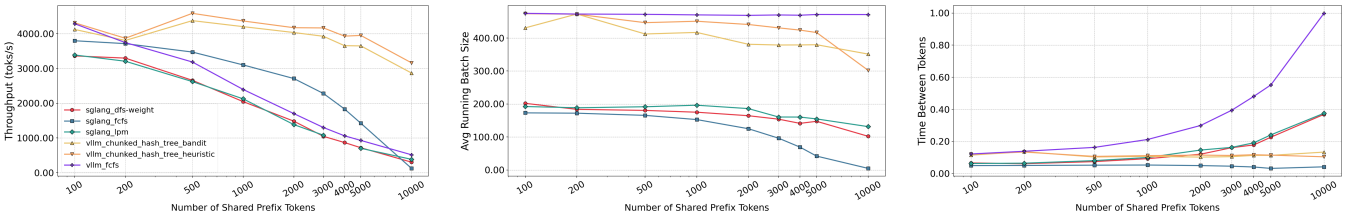
(a) (b) (c)

Figure 19. Various metrics across varying number of prefix groups. Shared prefix length of 5000 tokens; 500 input request poisson rate and 200 tokens decode length.



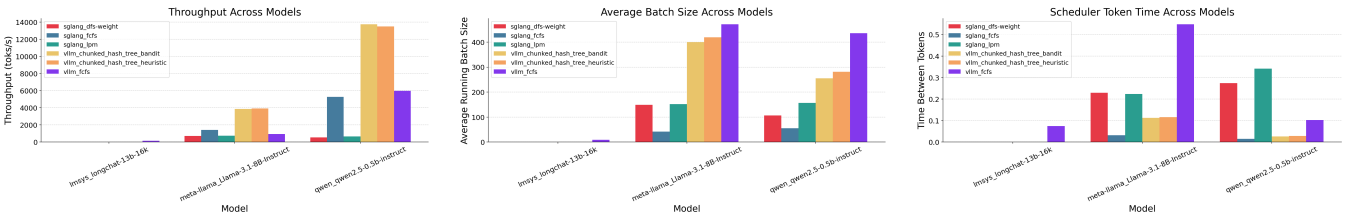
(a) (b) (c)

Figure 20. Various metrics across varying number of total decode tokens. Shared prefix length of 5000 tokens; 500 input request poisson rate and 5 prefix groups.



(a) (b) (c)

Figure 21. Various metrics across varying number of shared prefix tokens. 200 tokens decode length; 500 input request poisson rate and 5 prefix groups.



(a) (b) (c)

Figure 22. Various metrics across varying models. 200 tokens decode length; 500 input request poisson rate, 5 prefix groups, 5000 shared prefix tokens.

[10] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. 2024. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. arXiv:2307.13304 [cs.LG] https://arxiv.org/abs/2307.13304

[11] Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, and Chaoning Zhang. 2024. A Complete Survey on LLM-based AI Chatbots. arXiv:2406.16937 [cs.CL] https://arxiv.org/abs/2406.16937

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG] https://arxiv.org/abs/2205.14135

[13] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[14] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. 2023. Towards Next-Generation Intelligent Assistants Leveraging LLM Techniques. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23). Association for Computing Machinery, New York, NY, USA, 5792–5793. doi:10.1145/3580305.3599572

[15] Google. 2024. Gemini. https://gemini.google.com.

[16] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[17] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-A <scp>ttention</scp> : Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25). ACM, 897–912. doi:10.1145/3676641.3715996

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] https://arxiv.org/abs/2309.06180

[19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles.

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 [cs.CL] https://arxiv.org/abs/2005.11401

[22] Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. How Long Can Open-Source LLMs Truly Promise on Context Length? https://lmsys.org/blog/2023-06-29-longchat

[23] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978 [cs.CL] https://arxiv.org/abs/2306.00978

[24] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. 2025. Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing. arXiv:2504.19516 [cs.DC] https://arxiv.org/abs/2504.19516

[25] NVIDIA Corporation. 2024. NVIDIA RTX 6000 Ada Generation Datasheet. https://resources.nvidia.com/en-us-briefcase-for-datasheets/proviz-print-rtx6000-1?ncid=no-ncid Accessed: January 25, 2026.

[26] OpenAI. 2024. ChatGPT. https://chatgpt.com.

[27] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn,

Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[28] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. In Proceedings of Machine Learning and Systems, M. Zaharia, G. Joshi, and Y. Lin (Eds.), Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/96894468eb44631a32d7ebd56f9892c7-Paper-Conference.pdf

[29] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. arXiv:2503.05248 [cs.DC] https://arxiv.org/abs/2503.05248

[30] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. arXiv preprint arXiv:2503.05248 (2025).

[31] Ethan Perez, Douwe Kiela, and Kyunghyun Cho. 2021. True Few-Shot Learning with Language Models. In Advances in Neural Information Processing Systems, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 11054–11070. https://proceedings.neurips.cc/paper_files/paper/2021/file/5c04925674920eb58467fb52ce4ef728-Paper.pdf

[32] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. arXiv:2405.04437 [cs.LG] https://arxiv.org/abs/2405.04437

[33] Qidong Su, Wei Zhao, Xin Li, Muralidhar Andoorveedu, Chenhao Jiang, Zhanda Zhu, Kevin Song, Christina Giannoula, and Gennady Pekhimenko. 2025. Seesaw: High-throughput LLM Inference via Model Re-sharding. arXiv:2503.06433 [cs.DC] https://arxiv.org/abs/2503.06433

[34] vLLM Project Contributors. 2024. Automatic Prefix Caching. https://docs.vllm.ai/en/stable/design/prefix_caching/. vLLM Documentation. Accessed: 2026-03-10.

[35] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-Learning. Machine Learning 8, 3-4 (1992), 279–292. doi:10.1007/BF00992698

[36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. arXiv:2206.07682 [cs.CL] https://arxiv.org/abs/2206.07682

[37] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. Transactions on Machine Learning Research (2022). https://openreview.net/forum?id=yzkSU5zdwD Survey Certification.

[38] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. arXiv:2404.09526 [cs.DC] https://arxiv.org/abs/2404.09526

[39] Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. 2025. DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference. arXiv:2404.00242 [cs.CL] https://arxiv.org/abs/2404.00242

[40] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444 [cs.LG] https://arxiv.org/abs/2405.16444

[41] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. arXiv:2402.15220 [cs.LG] https://arxiv.org/abs/2402.15220

[42] Jinjun Yi, Zhixin Zhao, Yitao Hu, Ke Yan, Weiwei Sun, Hao Wang, Laiping Zhao, Yuhao Zhang, Wenxin Li, and Keqiu Li. 2025. PAT: Accelerating LLM Decoding via Prefix-Aware Attention with Resource Efficient Multi-Tile Kernel. arXiv:2511.22333 [cs.DC] https://arxiv.org/abs/2511.22333

[43] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. 2025. Jenga: Effective Memory Management for Serving LLM with Heterogeneity. arXiv:2503.18292 [cs.DC] https://arxiv.org/abs/2503.18292

[44] Yilong Zhao, Shuo Yang, Kan Zhu, Lianmin Zheng, Baris Kasikci, Yang Zhou, Jiarong Xing, and Ion Stoica. 2024. BlendServe: Optimizing Offline Inference for Auto-regressive Large Models with Resource-aware Batching. arXiv:2411.16102 [cs.LG] https://arxiv.org/abs/2411.16102

[45] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104

[46] Wanyi Zheng, Minxian Xu, Shengye Song, and Kejiang Ye. 2025. BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving. arXiv:2507.17120 [cs.DC] https://arxiv.org/abs/2507.17120

[47] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2025. BatchLLM: Optimizing Large Batched LLM Inference with Global Prefix Sharing and Throughput-oriented Token Batching.

arXiv:2412.03594 [cs.CL] https://arxiv.org/abs/2412.03594

[48] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670 [cs.DC] https://arxiv.org/abs/2401.09670

A Algorithms

B Throughput vs Batch Size for small requests with no prefix shared

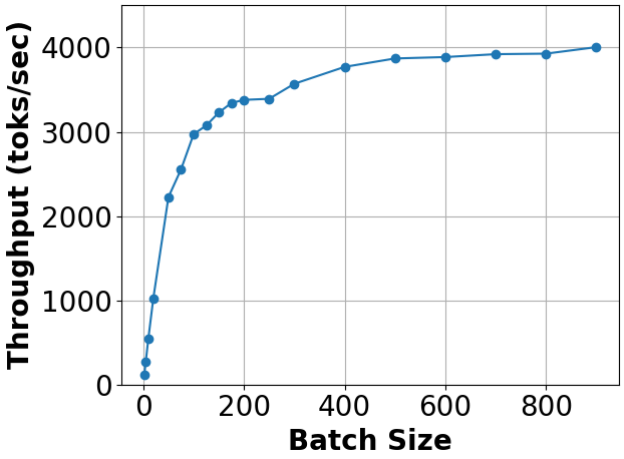


Figure 23. Throughput vs Batch Size

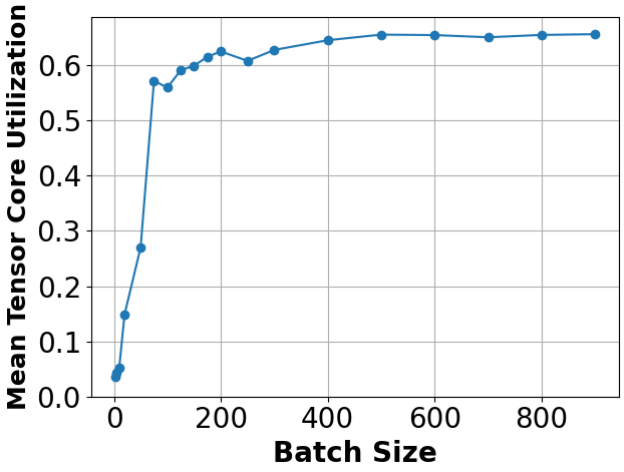


Figure 24. Tensor Core Utilization vs Batch Size

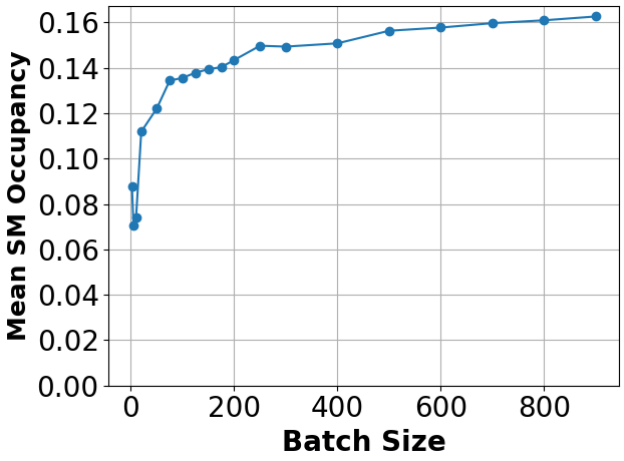


Figure 25. Mean SM Occupancy vs Batch Size

C Impact of Sharing Position on Decode Throughput - $5L$

To further strengthen our observations, we also present results for a sequence length of $5L$ in Figure 26, where we extend the length of the top shared nodes by L for all of these settings (the no sharing patterns remain unchanged, with both prefix groups of length $5L$ each). plot in appendix, but do we need this here?

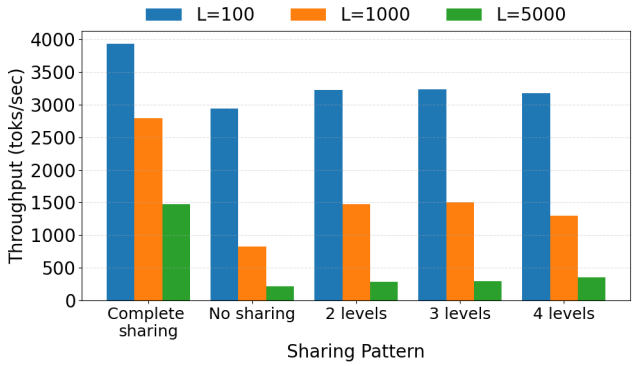


Figure 26. Throughput vs sharing patterns

D Homo vs Hetero - No Warmup - Varying Decode Tokens

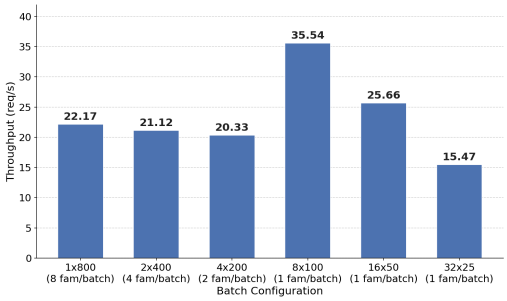


Figure 27. with warmup - 50 decode tokens

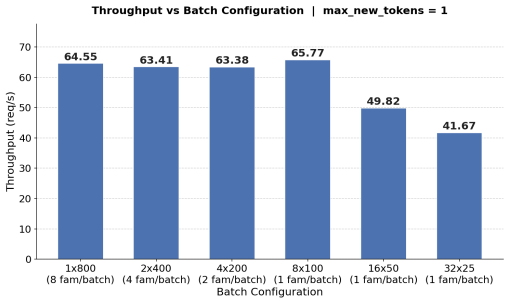


Figure 28. Throughput vs Batch Configuration

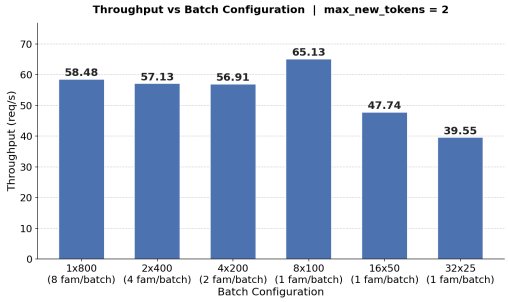


Figure 29. Throughput vs Batch Configuration

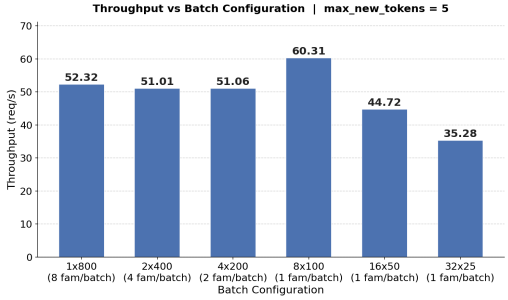


Figure 30. Throughput vs Batch Configuration

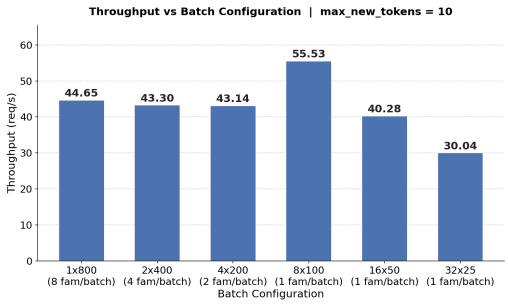


Figure 31. Throughput vs Batch Configuration

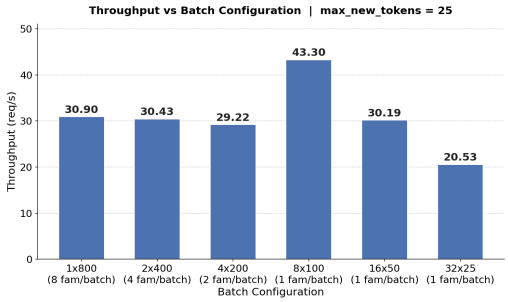


Figure 32. Throughput vs Batch Configuration

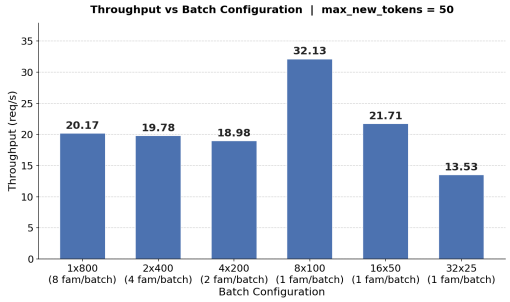


Figure 33. Throughput vs Batch Configuration

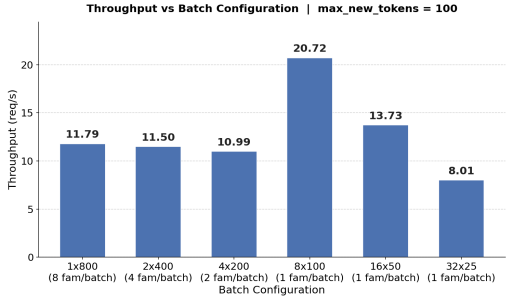


Figure 34. Throughput vs Batch Configuration

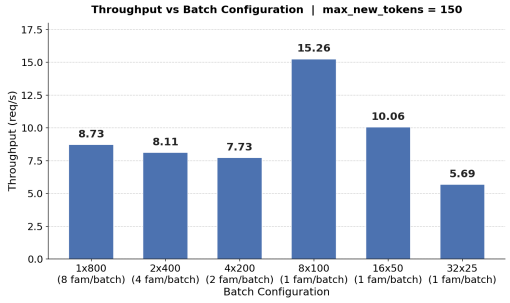


Figure 35. Throughput vs Batch Configuration

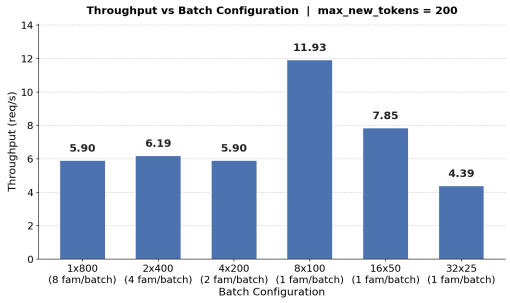


Figure 36. Throughput vs Batch Configuration

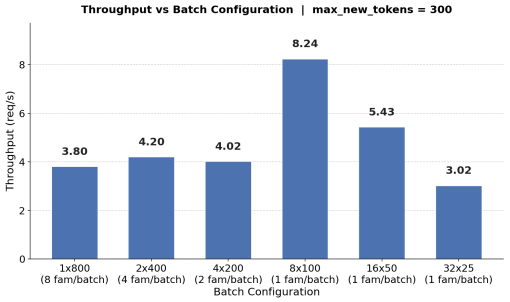


Figure 37. Throughput vs Batch Configuration

E Mean SM Occupancy vs Batch Size

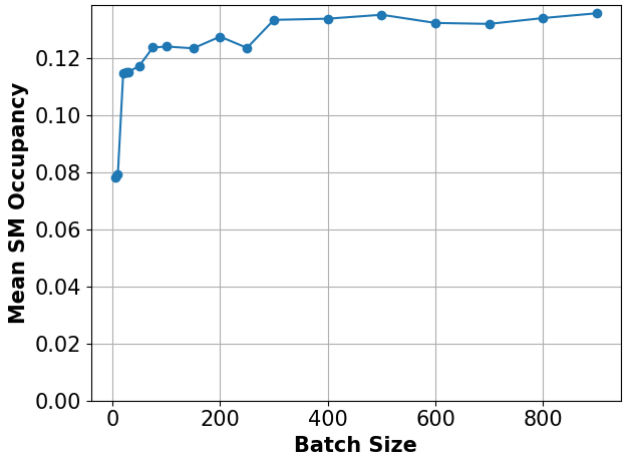


Figure 38. SM Occupancy

F Cache Working Set Size Theory

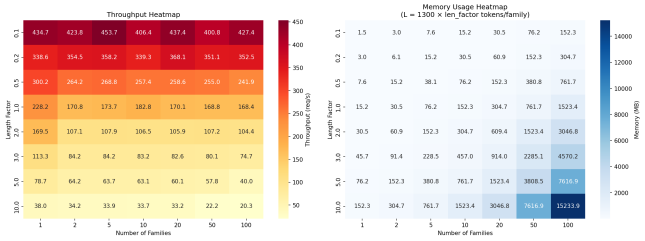
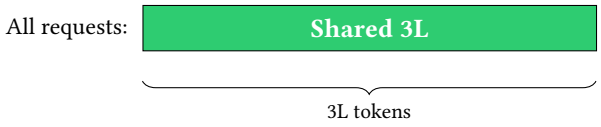


Figure 39. QWEN Throughput and KV Cache Memory across various sequence lengths and number of families

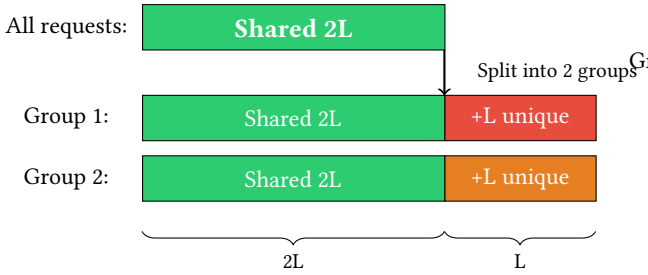
DUMP APPENDIX STARTS HERE!!!

G Radix Tree Workload Modes

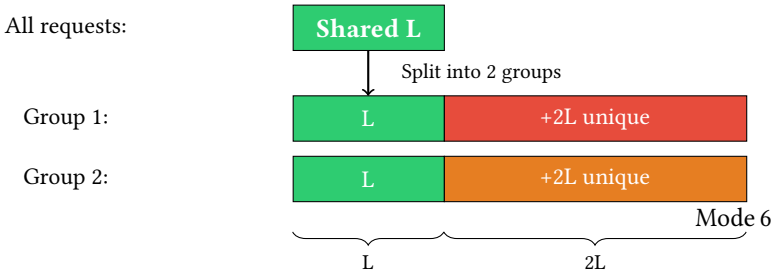
Full Sharing (All share 3L)



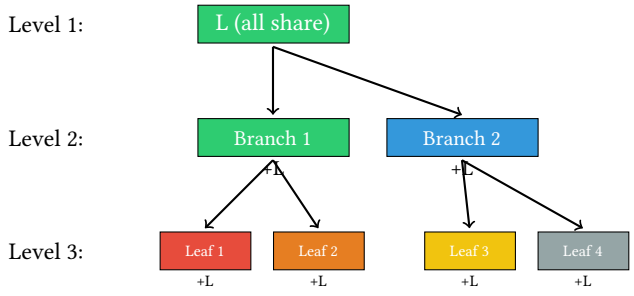
All share 2L, then 2xL



All share L, then 2x2L



Three-Level Tree (L → 2L → 3L, 4 leaves)



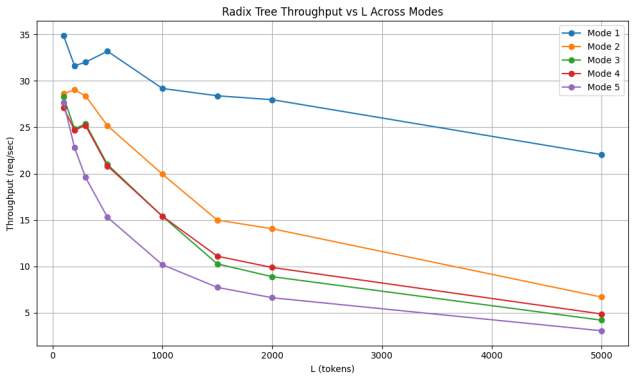
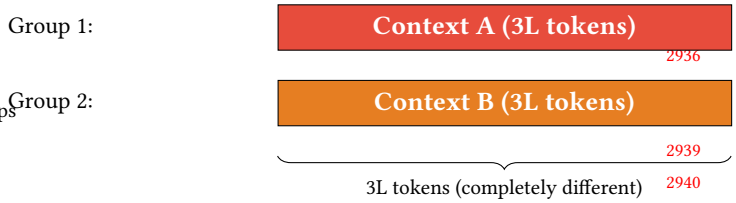
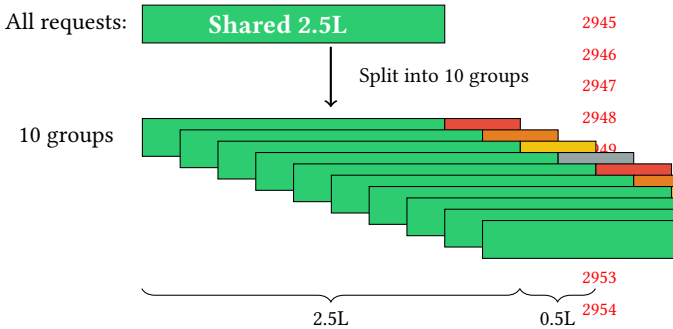


Figure 40. Results across varying L

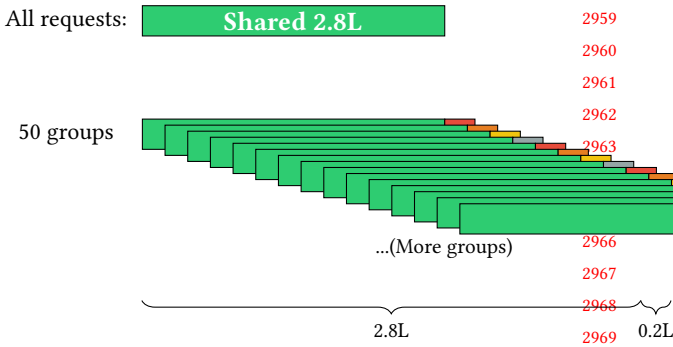
2 separate groups with 3L each



2.5L shared + 10x0.5L

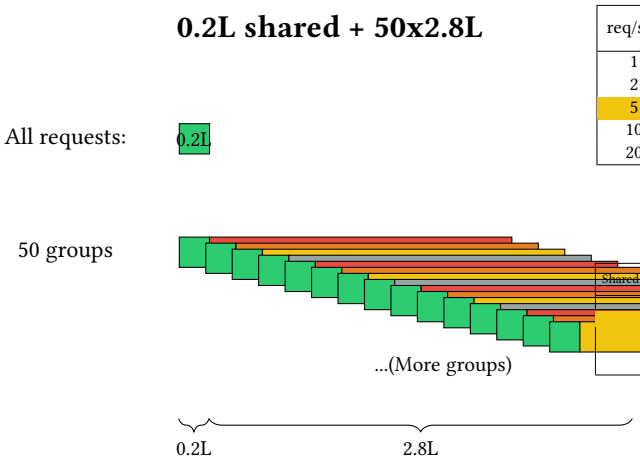


2.8L shared + 50x0.2L



Mode 6

Mode 7



req/sec	vLLM Baseline (FCFS)		vLLM (User-provided Family IDs)	
	Average Batch Size	Throughput	Average Batch Size	Throughput
1	2.6	1.04	2.02	0.95
2	5.15	2.00	4.48	1.99
5	29.60	5.09	11.11	4.40
10	395.80	4.42	23.96	8.16
20	438.85	5.25	52.79	13.32

Shared Prefix Length in Number of Tokens	vLLM Baseline (FCFS)		vLLM (User-provided Family IDs)	
	Average Batch Size	Throughput	Average Batch Size	Throughput
13	455.48	42.47	141.23	44.67
65	448.87	40.14	318.60	43.96
130	460.09	38.19	330.49	41.99
260	462.80	35.34	326.36	44.58
650	469.12	27.47	335.21	42.00

Mode 8

Comparison of Throughput across different modes

Model Name	vLLM Baseline (FCFS)		vLLM (User-provided Family IDs)	
	Average Batch Size	Throughput	Average Batch Size	Throughput
Qwen2.5-0.5B	378.98	25.40	89.63	49.08
Llama3.1-8B	448.87	4.34	151.21	30.84
LongChat-13B	5.28	0.61	215.70	22.01

Modes	**L = 1000**	**L = 2000**
Mode 1 - All share 3L	29.18	27.97
Mode 2 - All share 2L, then 2xL	19.94	14.06
Mode 3 - All share L, then 2x2L	15.40	8.89
Mode 4 - L → 2L → 3L, 4 leaves	15.41	9.88
Mode 5 - 2 separate groups with 3L each	10.18	6.62
Mode 6 - 2.5L shared + 10x0.5L	25.34	20.27
Mode 7 - 2.8L shared + 50x0.2L	28.05	23.17
Mode 8 - 0.2L shared + 50x2.8L	10.05	6.48

Table 3. Comparison of Throughput across different modes

H Finding the Sweet Zone

How many families?

- A high request arrival rate (500 req/sec)
- Max batch size of 500
- A shared prefix of length 6500 tokens
- 100 new output tokens
- Model: Llama3.1-8B

How many req/sec?

- 5 families
- Max batch size of 500
- A shared prefix of length 6500 tokens
- 100 new output tokens
- Model: Llama3.1-8B

How long should the shared prefix be?

- A high request arrival rate (500 req/sec)
- Max batch size of 500
- 5 families
- 100 new output tokens

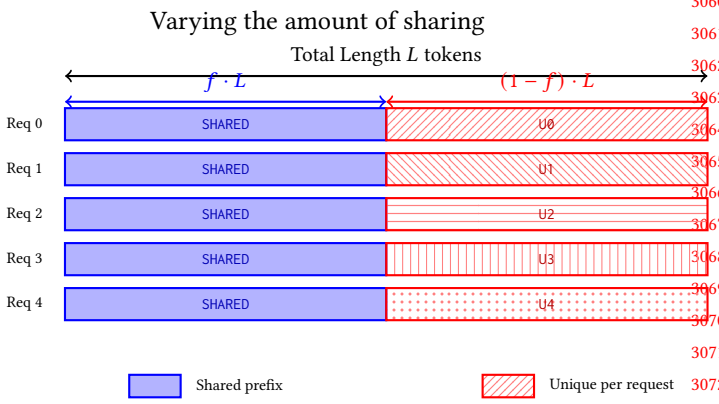
Number of Families	vLLM Baseline (FCFS)		vLLM (User-provided Family IDs)	
	Average Batch Size	Throughput	Average Batch Size	Throughput
10	465.48	5.92	141.23	21.94
20	455.47	5.74	72.67	15.27
50	57.48	2.07	64.77	14.84
100	41.97	1.11	67.14	14.82

- Model: Llama3.1-8B

How large should the model be?

- A high request arrival rate (500 req/sec)
- Max batch size of 500
- 5 families
- 100 new output tokens
- Shared prefix length: 6500 tokens

I Other experiments



Varying the amount of sharing (L = 100 tokens)
Varying the amount of sharing (L = 500 tokens)
Varying the amount of sharing (L = 1000 tokens)
Varying the amount of sharing (L = 2000 tokens)
Varying the amount of sharing (L = 500 tokens - More points)

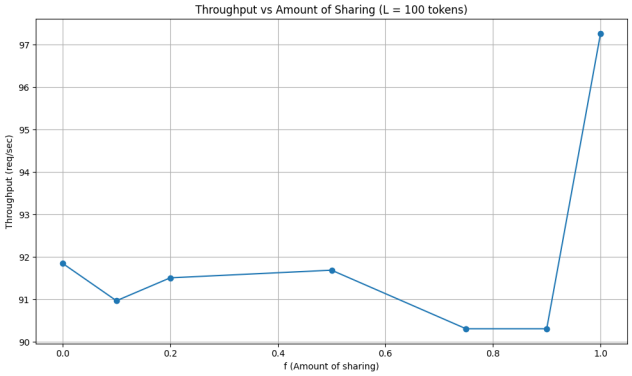


Figure 41. Results across varying L

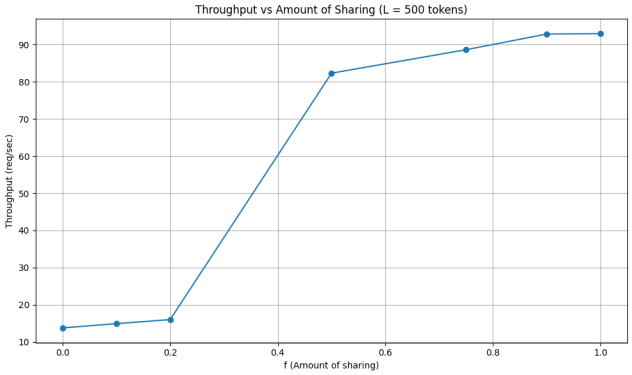


Figure 42. Results across varying L

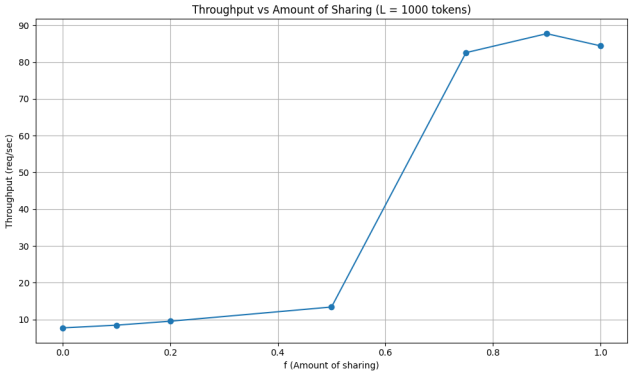
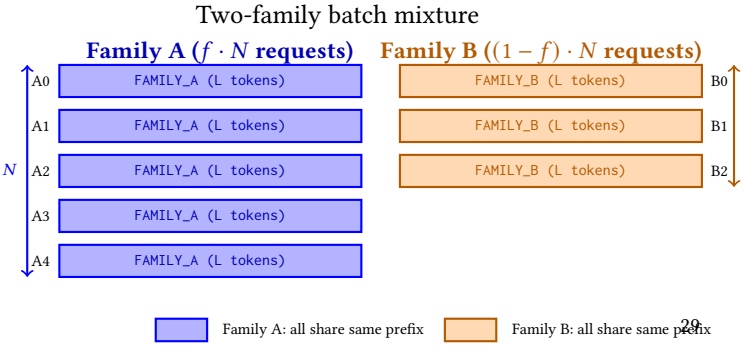


Figure 43. Results across varying L



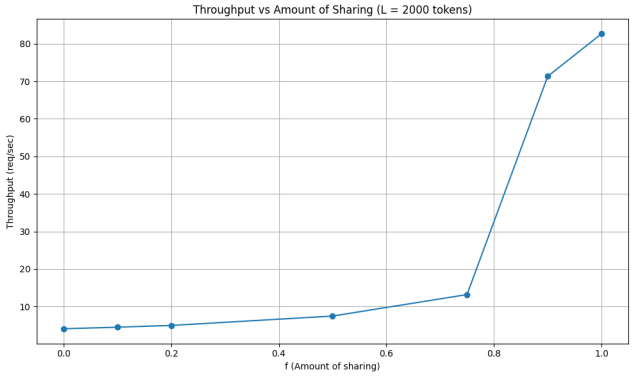


Figure 44. Results across varying L

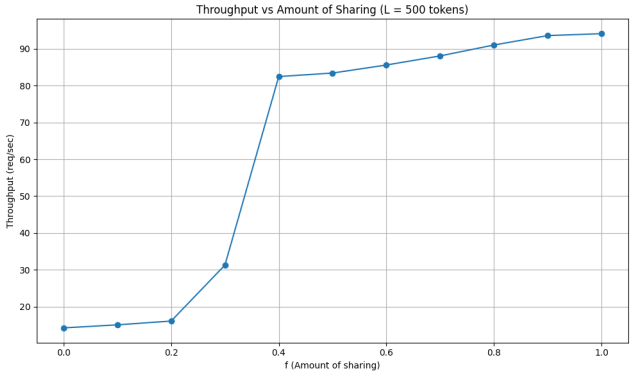


Figure 45. Results across varying L

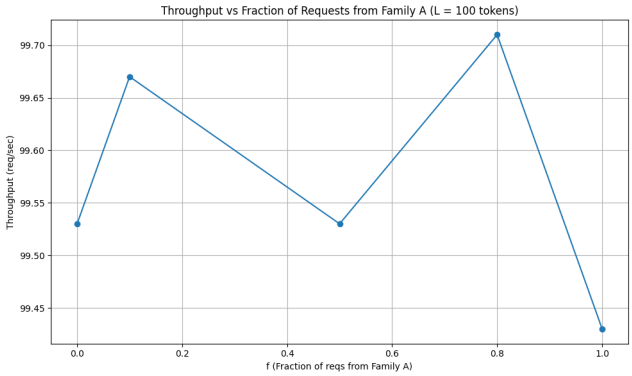


Figure 46. Results across fraction of requests from A

Two Family Batch Mixture (L = 100 tokens)
Two Family Batch Mixture (L = 500 tokens)
Two Family Batch Mixture (L = 1000 tokens)
Two Family Batch Mixture (L = 2000 tokens)

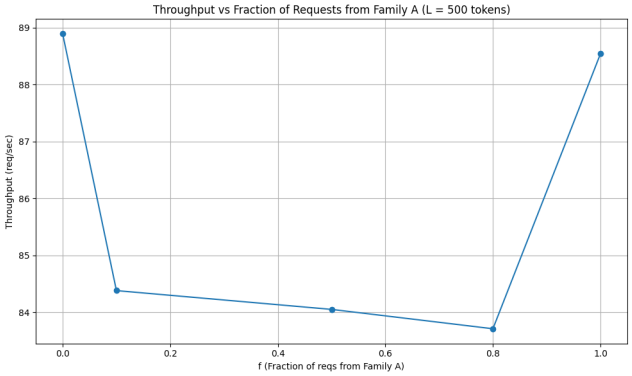


Figure 47. Results across fraction of requests from A

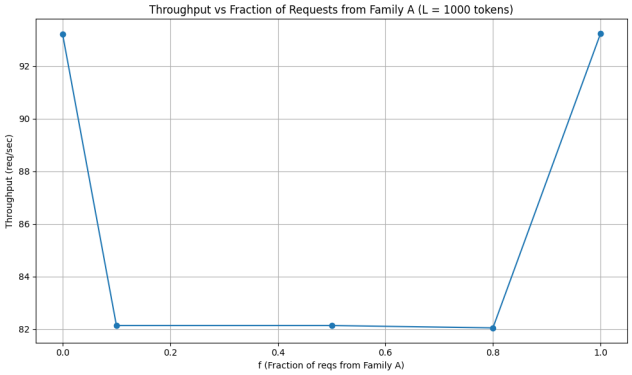


Figure 48. Results across fraction of requests from A

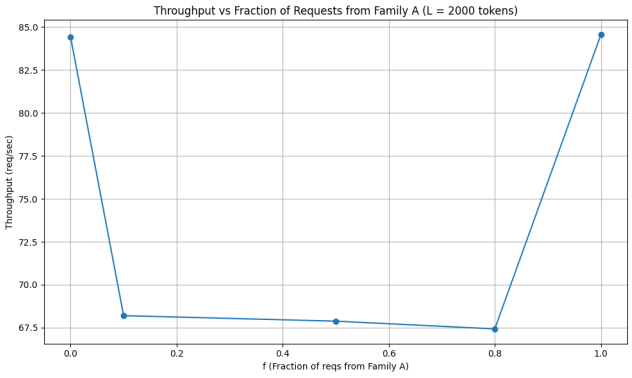


Figure 49. Results across fraction of requests from A

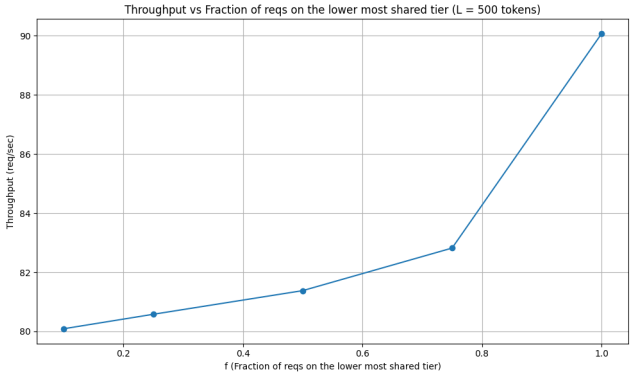


Figure 50. Varying fraction on lowermost tier

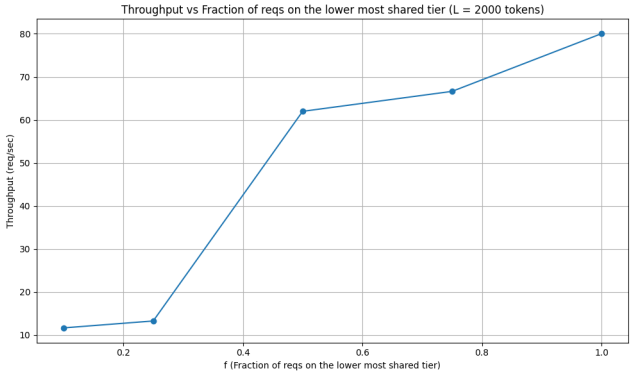


Figure 51. Varying fraction on lowermost tier

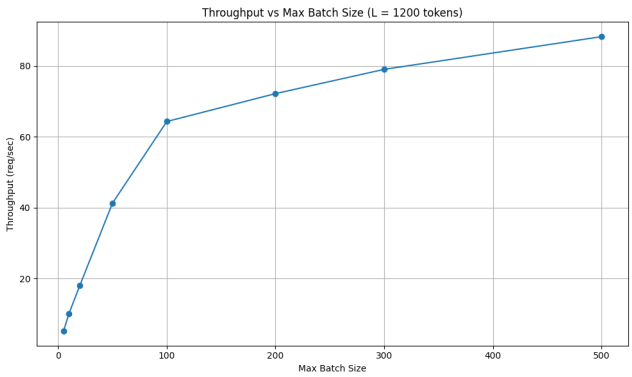
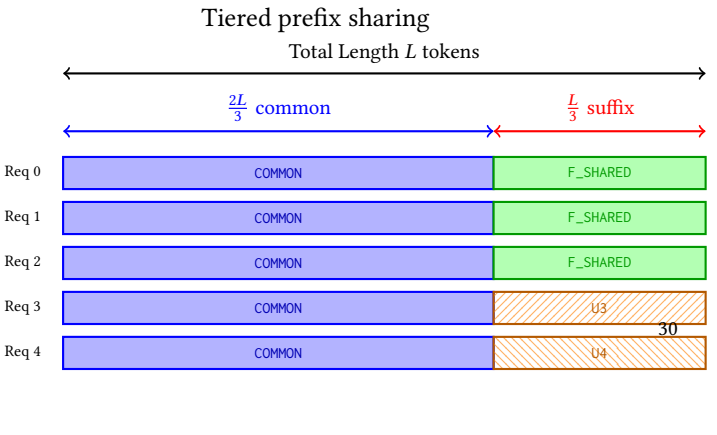


Figure 52. Throughput vs Max Batch Size (L = 1200 tokens)



Tiered Prefix Sharing (L = 500 tokens)
Tiered Prefix Sharing (L = 2000 tokens)
Throughput vs Max Batch Size
Two Family Mixture - Mean DRAM Activity
Two Family Mixture - Total DRAM Accesses
Two Family Mixture - Mean SM Activity
Two Family Mixture - Mean SM Occupancy
Two Family Mixture - Mean Tensor Core Utilization

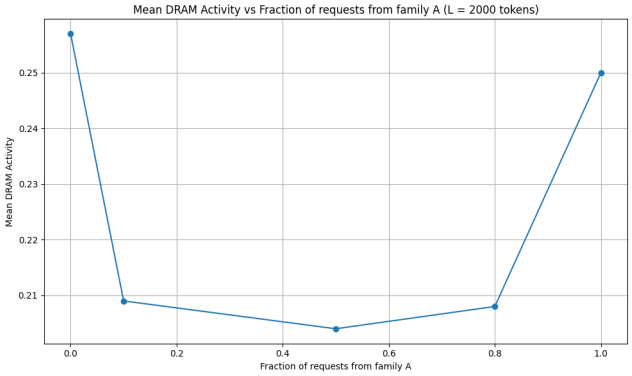


Figure 53. Mean DRAM Activity across fraction of requests from A (L = 2000 tokens)

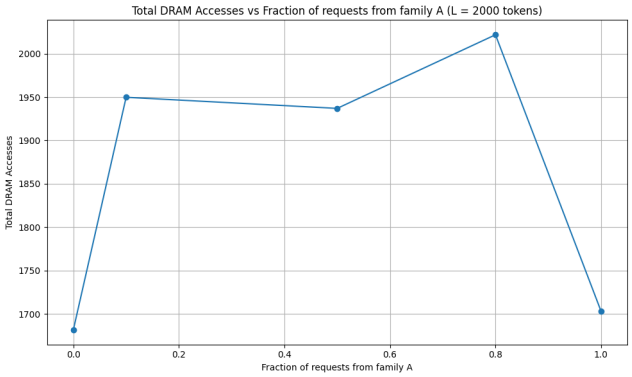


Figure 54. Total DRAM Accesses across fraction of requests from A (L = 2000 tokens)

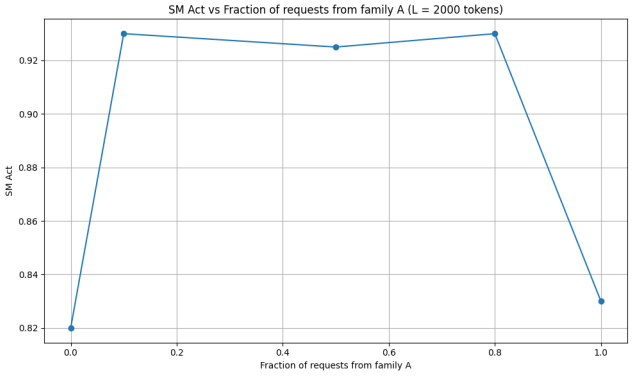
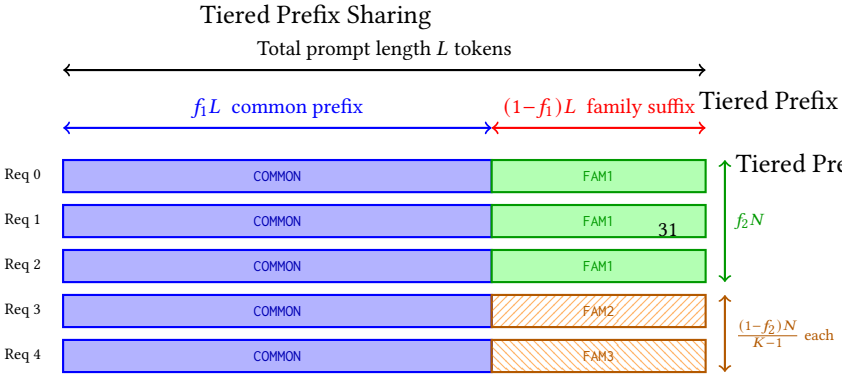


Figure 55. Mean SM Activity across fraction of requests from A (L = 2000 tokens)



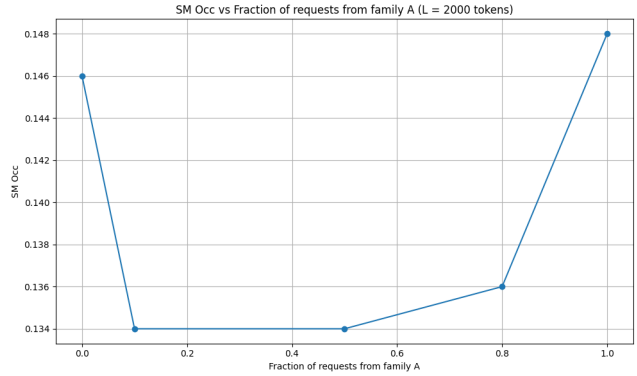


Figure 56. Mean SM Occupancy across fraction of requests from A (L = 2000 tokens)

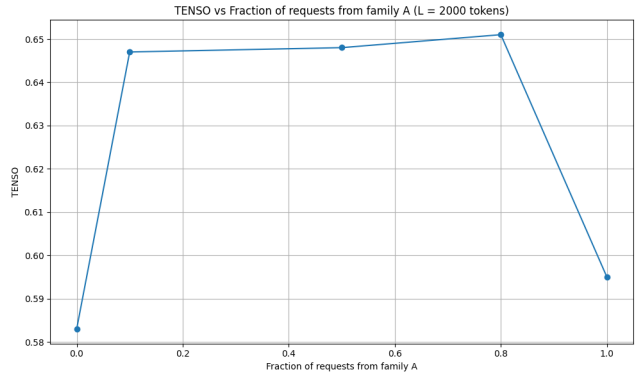
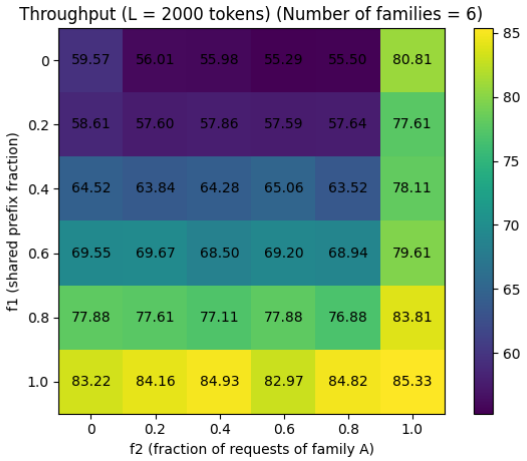
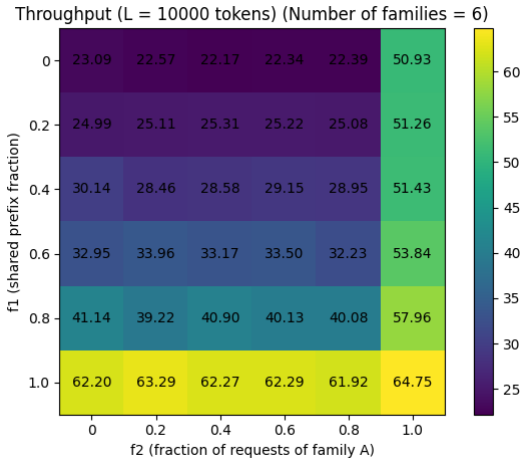


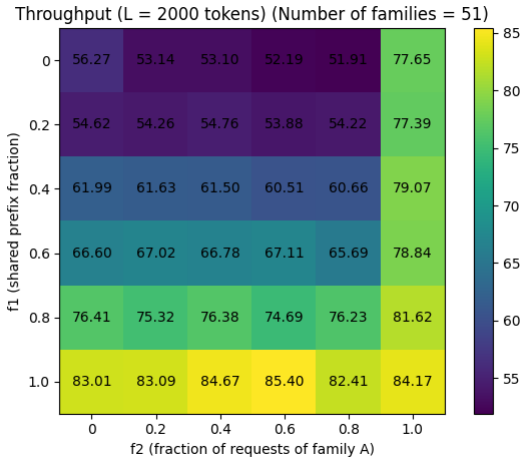
Figure 57. Mean Tensor Core Utilization across fraction of requests from A (L = 2000 tokens)

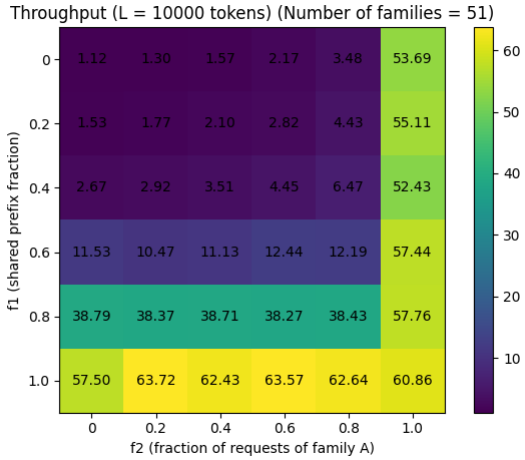


Tiered Prefix Sharing (L = 2000 tokens) (Number of families = 6)

Tiered Prefix Sharing (L = 10000 tokens) (Number of families = 6)







Tiered Prefix Sharing (L = 2000 tokens) (Number of families = 51)

Tiered Prefix Sharing (L = 10000 tokens) (Number of families = 51)

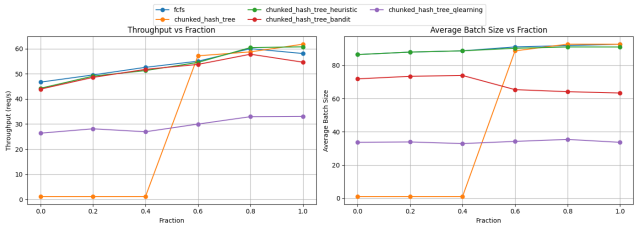
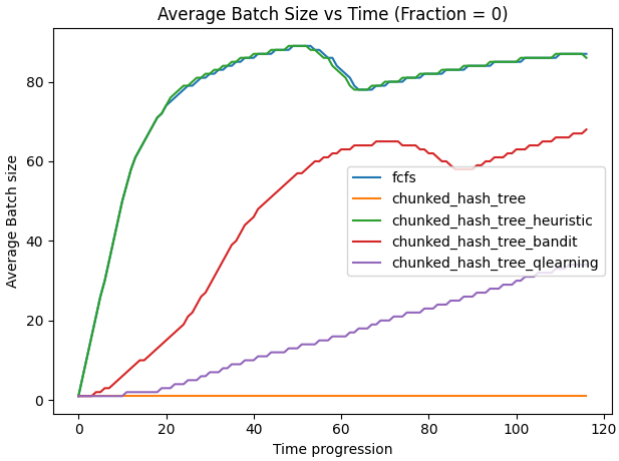
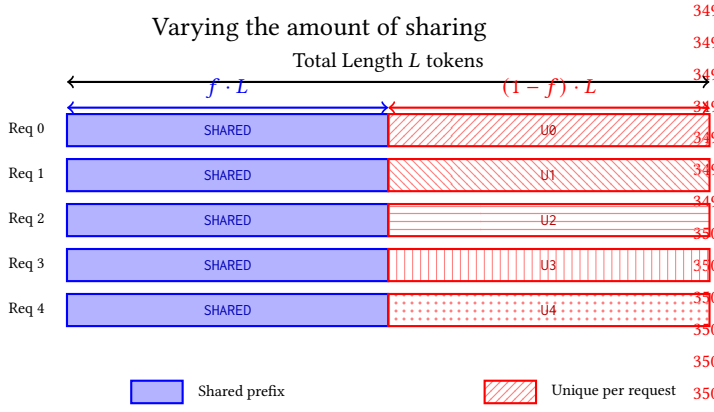


Figure 58. Throughput and Average Batch Sizes across Fractions ($L = 1200$ tokens, batch of 100 requests)





Varying the amount of sharing - different RL Techniques

Varying the amount of sharing - different RL Techniques - average batch size (fraction = 0)

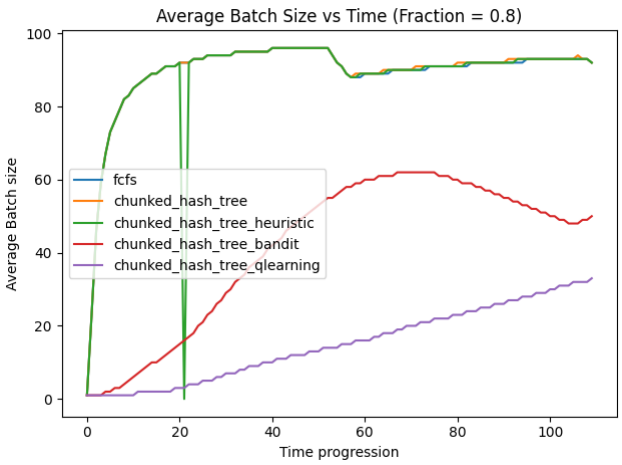
Varying the amount of sharing - different RL Techniques - average batch size (fraction = 0.2)

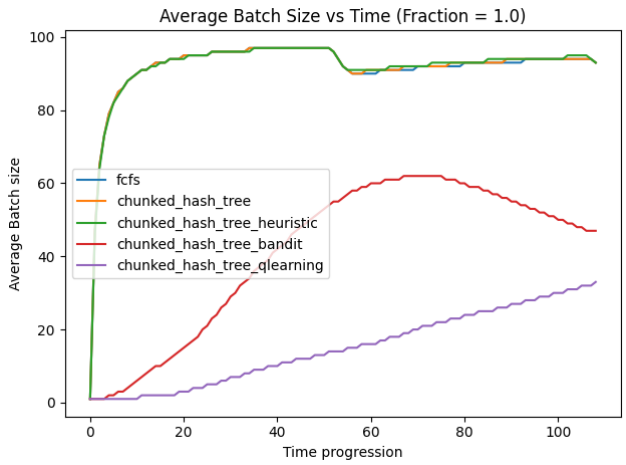
Varying the amount of sharing - different RL Techniques - average batch size (fraction = 0.4)

Varying the amount of sharing - different RL Techniques - average batch size (fraction = 0.6)

Varying the amount of sharing - different RL Techniques - average batch size (fraction = 0.8)

Varying the amount of sharing - different RL Techniques - average batch size (fraction = 1.0)


Average Batch Size vs Time (Fraction = 0.8)
Average Batch size
Time progression
fcfs
chunked_hash_tree
chunked_hash_tree_heuristic
chunked_hash_tree_bandit
chunked_hash_tree_qlearning
0
20
40
60
80
100


Average Batch Size vs Time (Fraction = 1.0)
Average Batch size
Time progression
fcfs
chunked_hash_tree
chunked_hash_tree_heuristic
chunked_hash_tree_bandit
chunked_hash_tree_qlearning
0
20
40
60
80
100